



DEPARTAMENTO
DE COMPUTACION

Facultad de Ciencias Exactas y Naturales - UBA

Lógica Digital

Circuitos secuenciales

Sistemas Digitales

2do Cuatrimestre 2026

FCEN - UBA

Introducción



Hoy vamos a ver los principios de diseño, práctica y ejemplos de circuitos secuenciales, la estructura de la clase va a ser la siguiente:

- **Latches**

Hoy vamos a ver los principios de diseño, práctica y ejemplos de circuitos secuenciales, la estructura de la clase va a ser la siguiente:

- **Latches**
- **Flip-flops**

Hoy vamos a ver los principios de diseño, práctica y ejemplos de circuitos secuenciales, la estructura de la clase va a ser la siguiente:

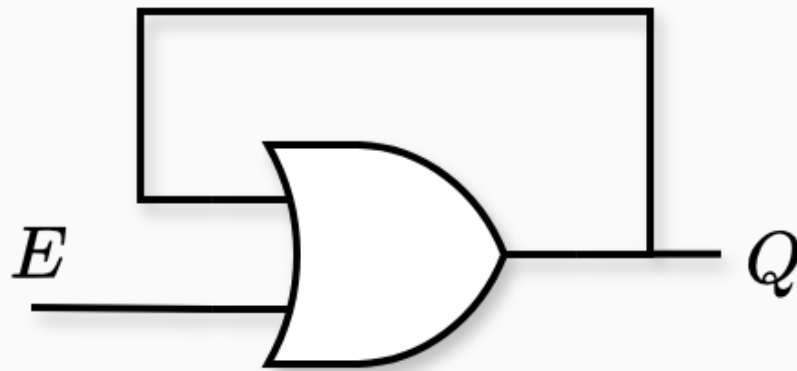
- **Latches**
- **Flip-flops**
- **Metaestabilidad**

Hoy vamos a ver los principios de diseño, práctica y ejemplos de circuitos secuenciales, la estructura de la clase va a ser la siguiente:

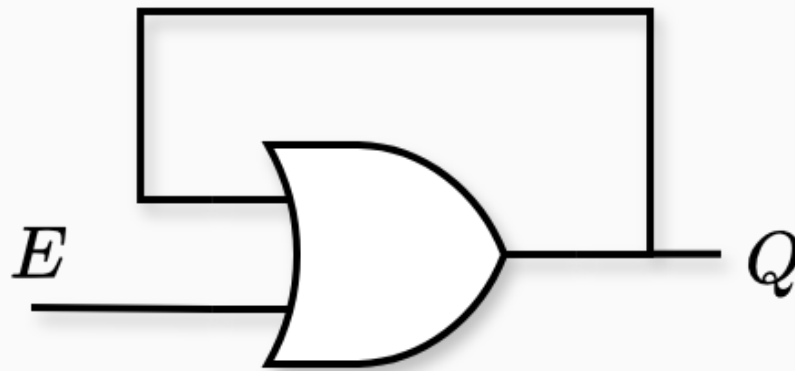
- **Latches**
- **Flip-flops**
- **Metaestabilidad**
- **Registros y memorias**

Latches

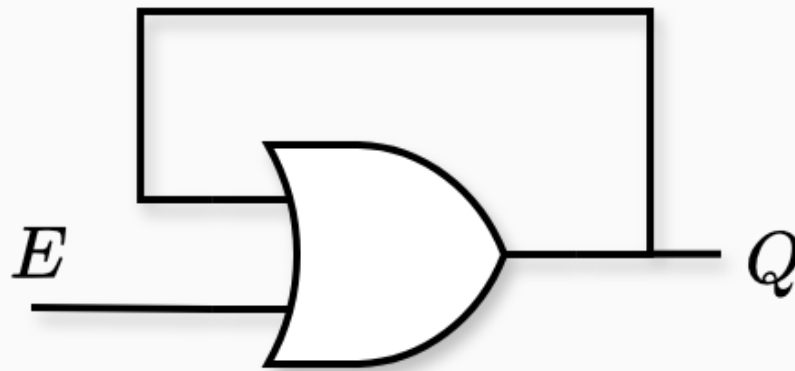




- Asumamos que tanto la entrada E como la salida Q valen 0 en este instante.

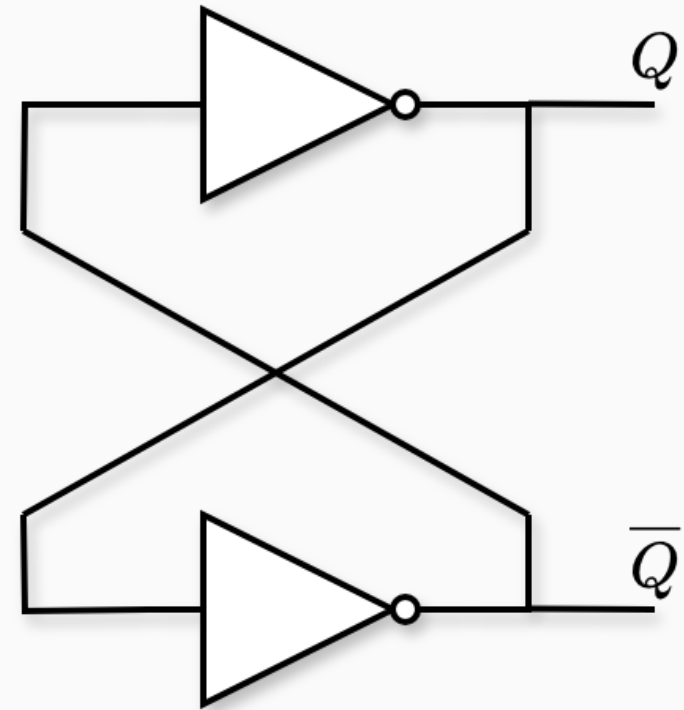


- Asumamos que tanto la entrada E como la salida Q valen 0 en este instante.
- ¿Qué pasa si ahora E cambia a 1?

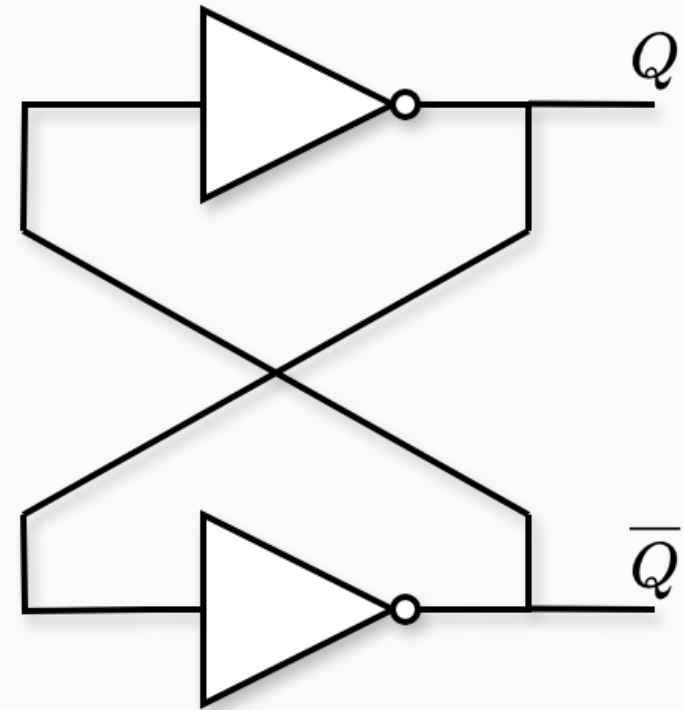


- Asumamos que tanto la entrada E como la salida Q valen 0 en este instante.
- ¿Qué pasa si ahora E cambia a 1?
- Y luego... ¿si E vuelve a 0?

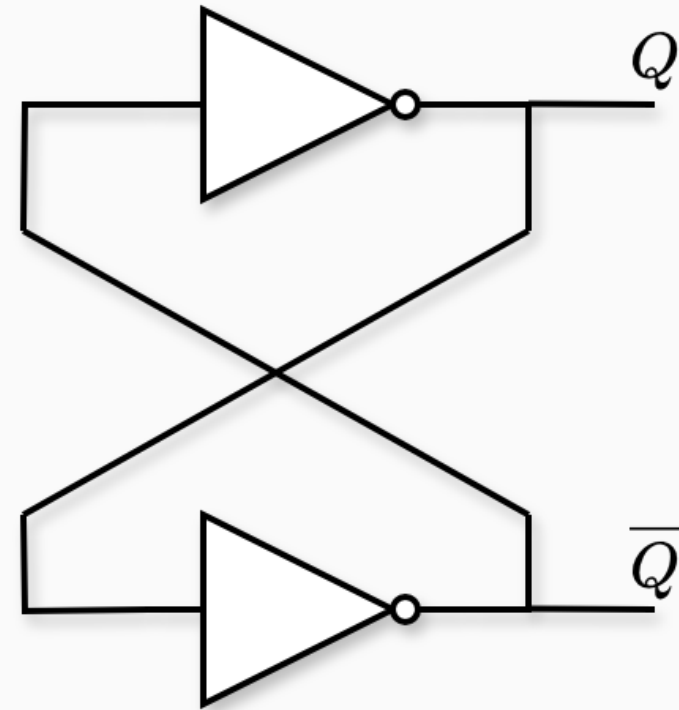
- Estabilidad 1: si $Q = 0$, entonces $\overline{Q} = 1$



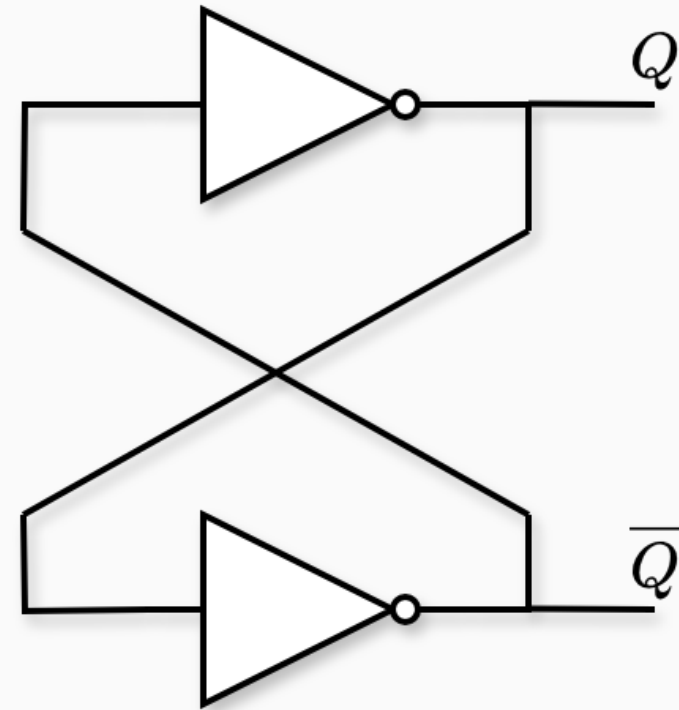
- Estabilidad 1: si $Q = 0$, entonces $\overline{Q} = 1$
- Estabilidad 2: si $Q = 1$, entonces $\overline{Q} = 0$



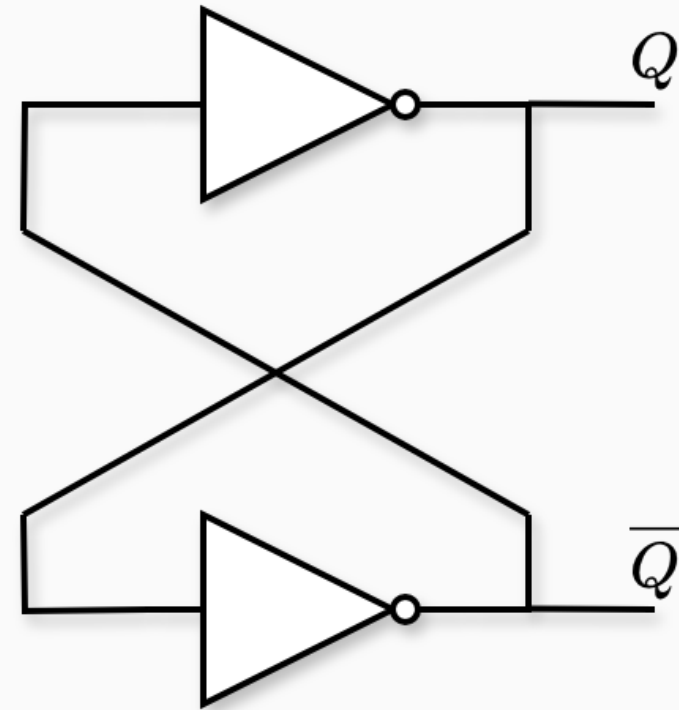
- Estabilidad 1: si $Q = 0$, entonces $\overline{Q} = 1$
- Estabilidad 2: si $Q = 1$, entonces $\overline{Q} = 0$
- De ahí el nombre de **biestable**



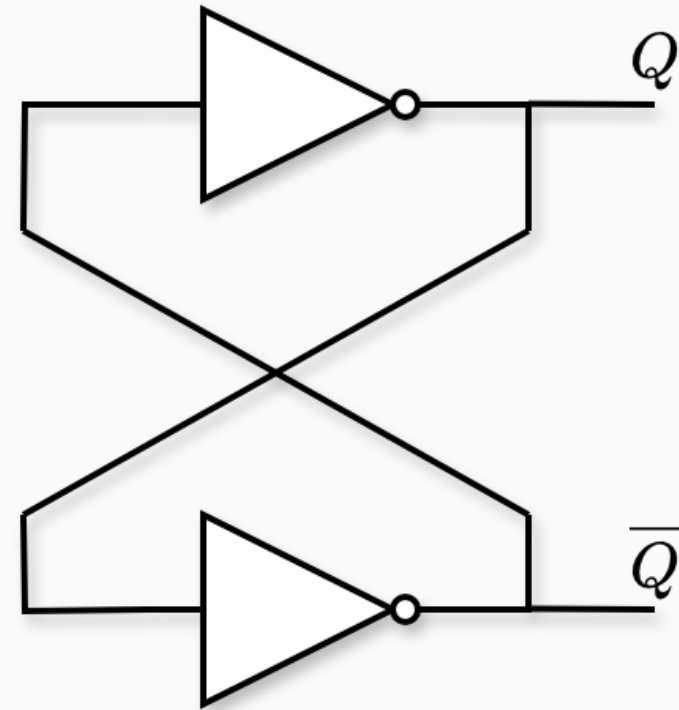
- Estabilidad 1: si $Q = 0$, entonces $\overline{Q} = 1$
- Estabilidad 2: si $Q = 1$, entonces $\overline{Q} = 0$
- De ahí el nombre de **bistable**
- Decimos que este circuito tiene **memoria**.



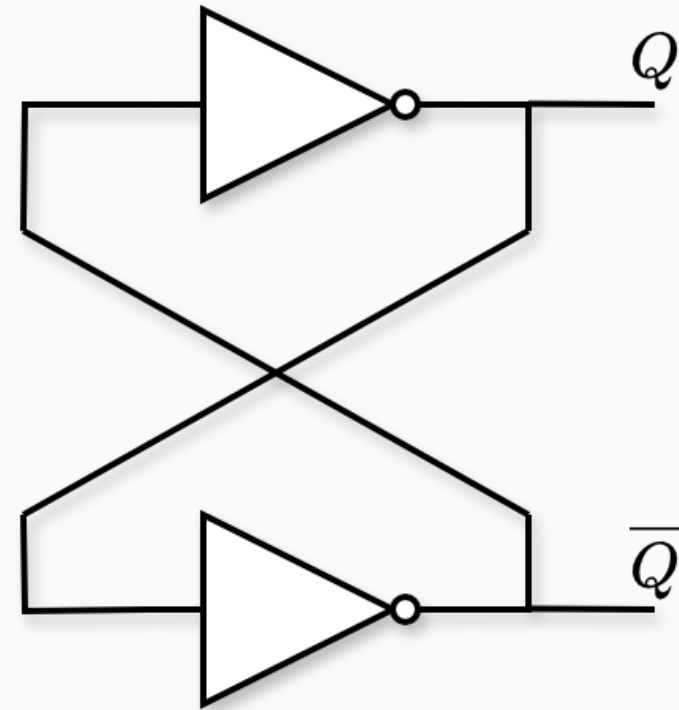
- Estabilidad 1: si $Q = 0$, entonces $\overline{Q} = 1$
- Estabilidad 2: si $Q = 1$, entonces $\overline{Q} = 0$
- De ahí el nombre de **bistable**
- Decimos que este circuito tiene **memoria**.
- **No** podemos controlar el estado de la memoria.



- Estabilidad 1: si $Q = 0$, entonces $\overline{Q} = 1$
- Estabilidad 2: si $Q = 1$, entonces $\overline{Q} = 0$
- De ahí el nombre de **bistable**
- Decimos que este circuito tiene **memoria**.
- **No** podemos controlar el estado de la memoria.



- Estabilidad 1: si $Q = 0$, entonces $\overline{Q} = 1$
- Estabilidad 2: si $Q = 1$, entonces $\overline{Q} = 0$
- De ahí el nombre de **bistable**
- Decimos que este circuito tiene **memoria**.
- **No** podemos controlar el estado de la memoria.

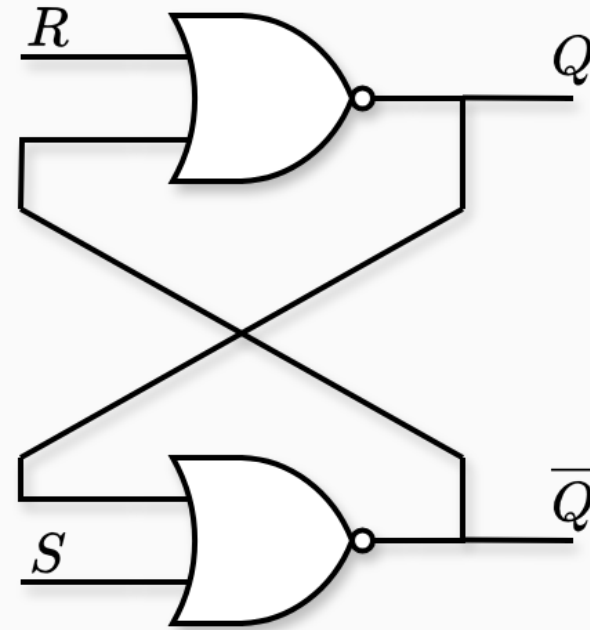


Hay un tercer estado *metaestable*, que veremos más adelante.



- Con un biestable, no podíamos controlar el estado de la memoria.

- Con un biestable, no podíamos controlar el estado de la memoria.
- Con un *SR latch*, podemos *setear* y *resetear* la salida Q .

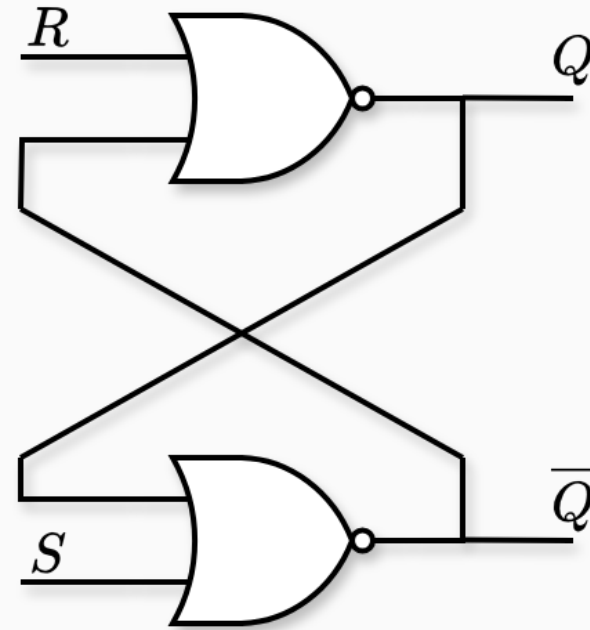


A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

- Con un biestable, no podíamos controlar el estado de la memoria.
- Con un *SR latch*, podemos *setear* y *resetear* la salida Q .

Tabla de verdad:

S	R	Q	$\overline{Q}$
1	0		
0	0		
0	1		
1	1		

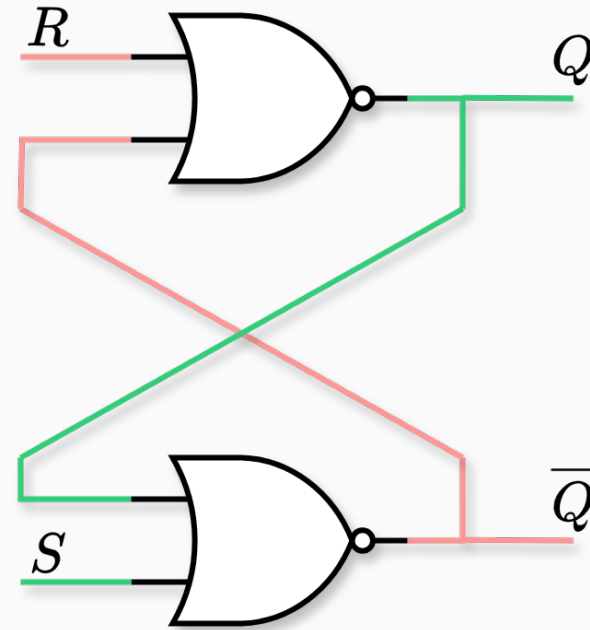


A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

- Con un biestable, no podíamos controlar el estado de la memoria.
- Con un *SR latch*, podemos *setear* y *resetear* la salida Q .

Tabla de verdad:

S	R	Q	$\overline{Q}$
1	0	1	0
0	0		
0	1		
1	1		

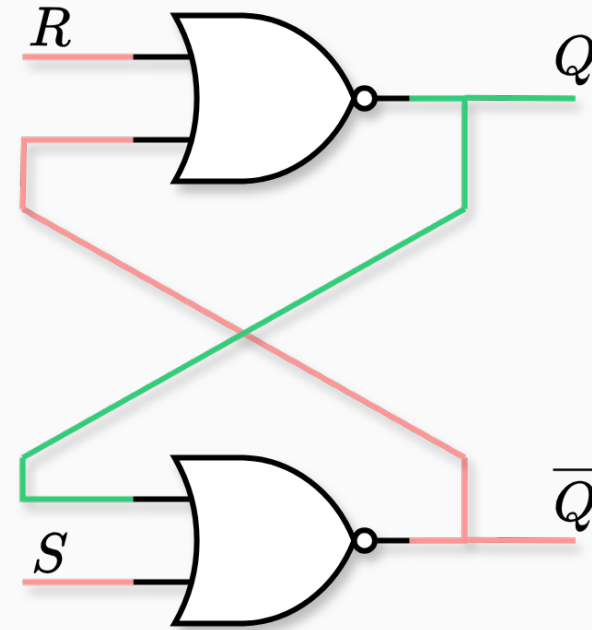


A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

- Con un biestable, no podíamos controlar el estado de la memoria.
- Con un *SR latch*, podemos *setear* y *resetear* la salida Q .

Tabla de verdad:

S	R	Q	$\overline{Q}$
1	0	1	0
0	0	Q_{prev}	$\overline{Q}_{\text{prev}}$
0	1		
1	1		

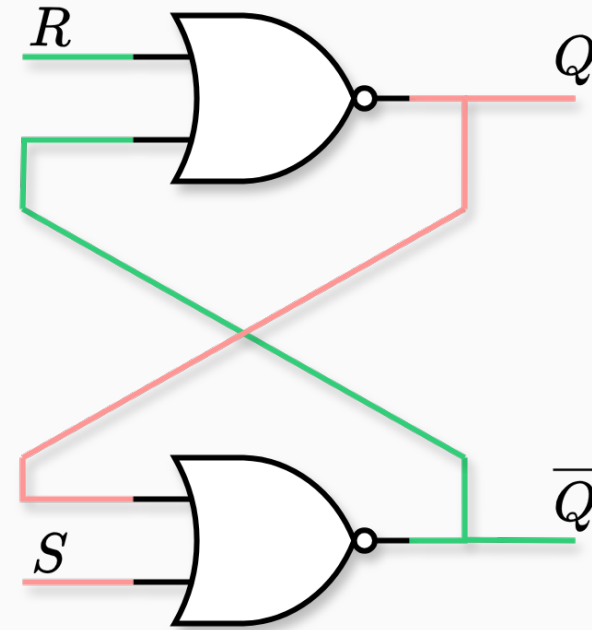


A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

- Con un biestable, no podíamos controlar el estado de la memoria.
- Con un *SR latch*, podemos *setear* y *resetear* la salida Q .

Tabla de verdad:

S	R	Q	$\overline{Q}$
1	0	1	0
0	0	Q_{prev}	$\overline{Q}_{\text{prev}}$
0	1	0	1
1	1		

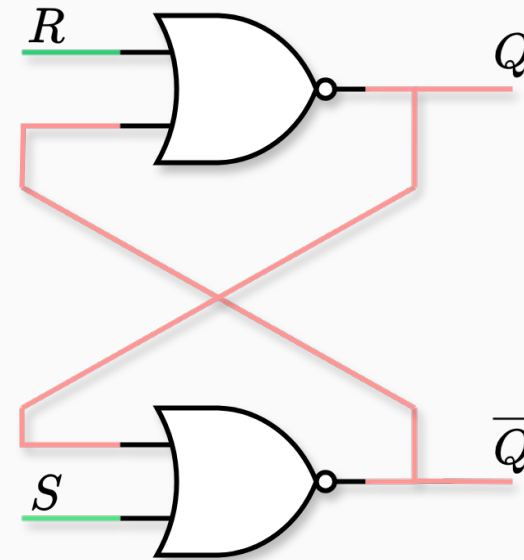


A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

- Con un biestable, no podíamos controlar el estado de la memoria.
- Con un *SR latch*, podemos *setear* y *resetear* la salida Q .

Tabla de verdad:

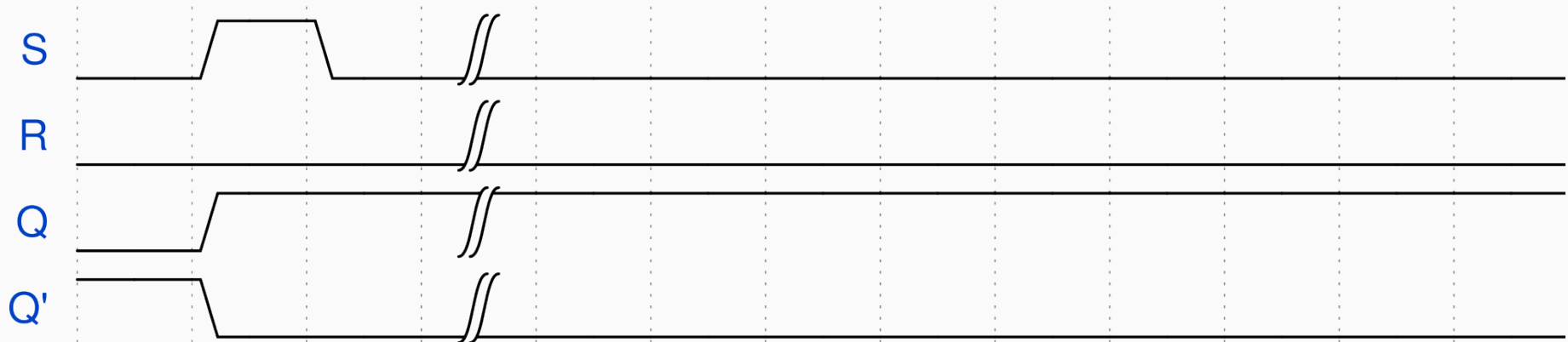
S	R	Q	$\overline{Q}$
1	0	1	0
0	0	Q_{prev}	$\overline{Q}_{\text{prev}}$
0	1	0	1
1	1	0	0



A	B	A NOR B
0	0	1
0	1	0
1	0	0
1	1	0

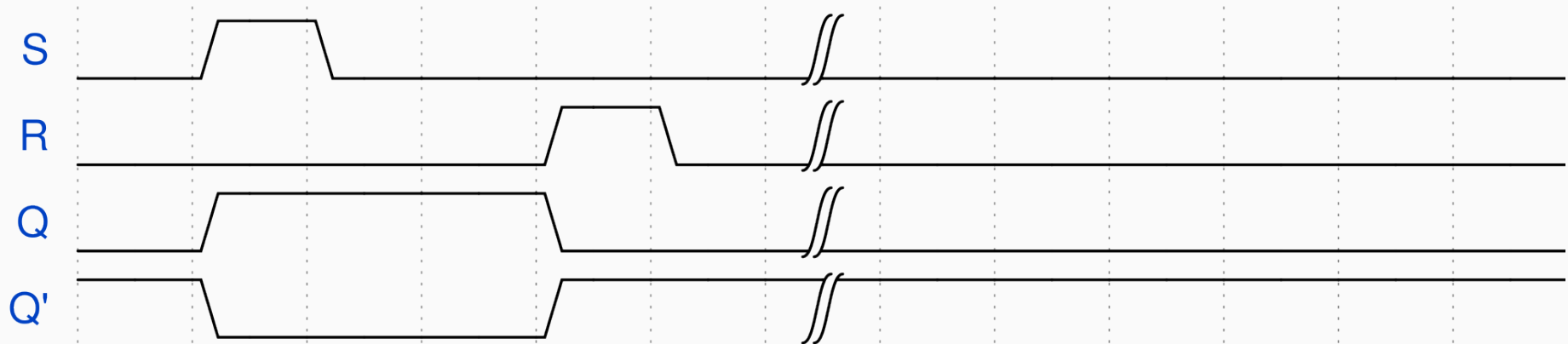
Latch SR

S	R	Q	$\overline{Q}$
0	0	latch	latch
0	1	0	1
1	0	1	0
1	1	U	U



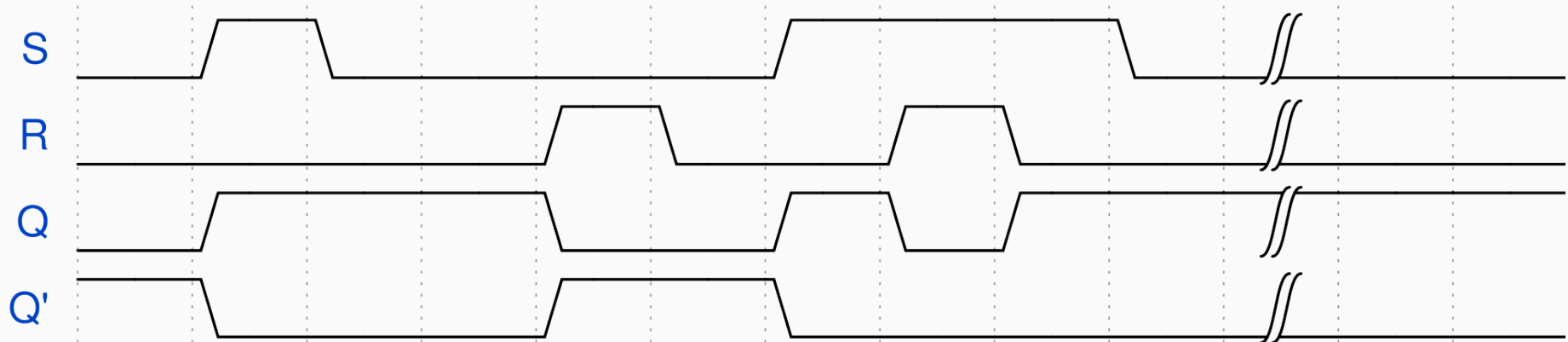
Latch SR

S	R	Q	$\overline{Q}$
0	0	latch	latch
0	1	0	1
1	0	1	0
1	1	U	U



Latch SR

S	R	Q	$\overline{Q}$
0	0	latch	latch
0	1	0	1
1	0	1	0
1	1	U	U

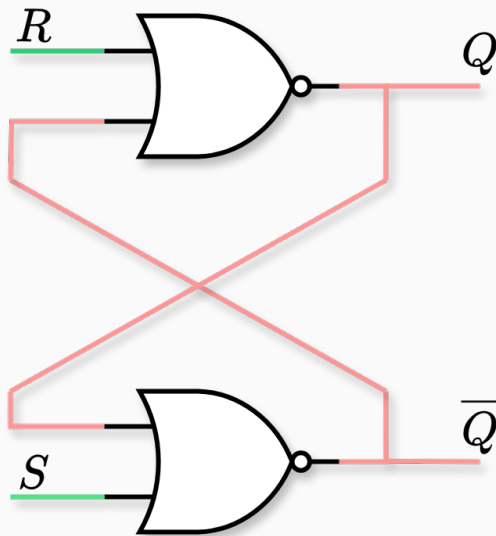


**DEPARTAMENTO
DE COMPUTACION**

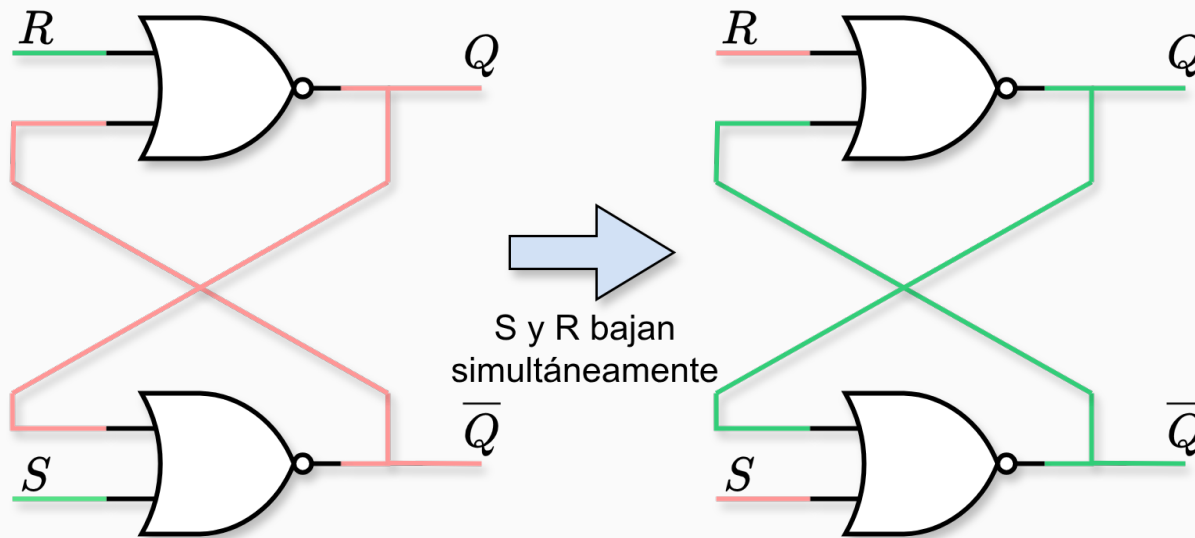
Facultad de Ciencias Exactas y Naturales - UBA

The timing diagram shows the behavior of a D flip-flop. The inputs S and R are active-low signals. The output Q is shaded with diagonal lines when it is high. The output Q' is the complement of Q. The diagram shows that the output Q follows the input D when S and R are both high. When S or R is low, the output Q is set or reset to a specific value.

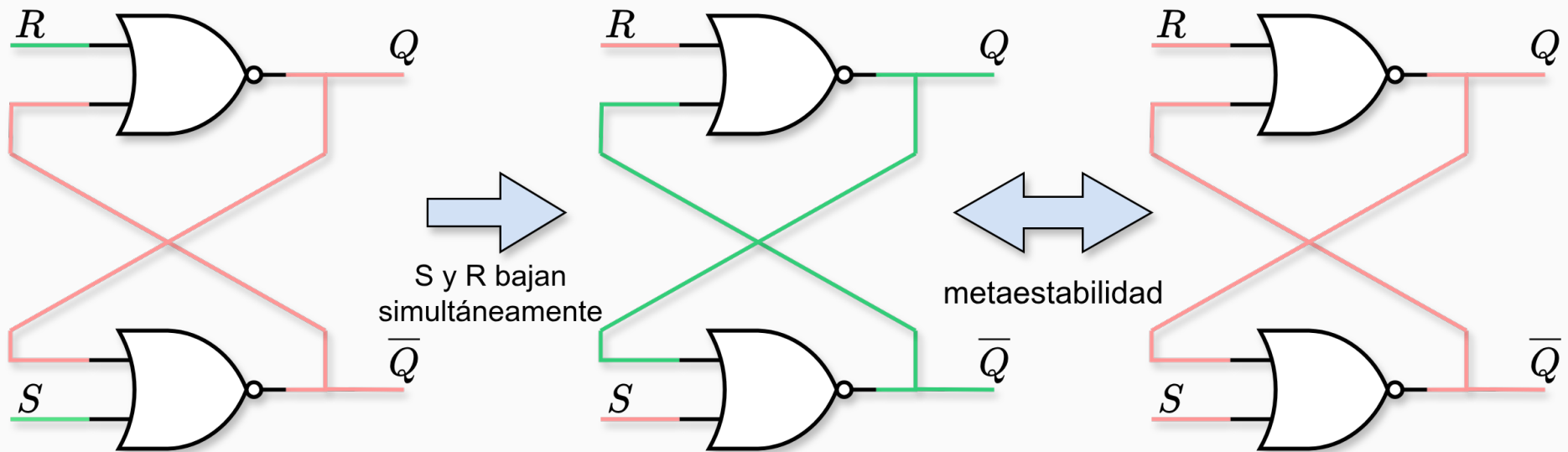
Latch SR. Metaestabilidad



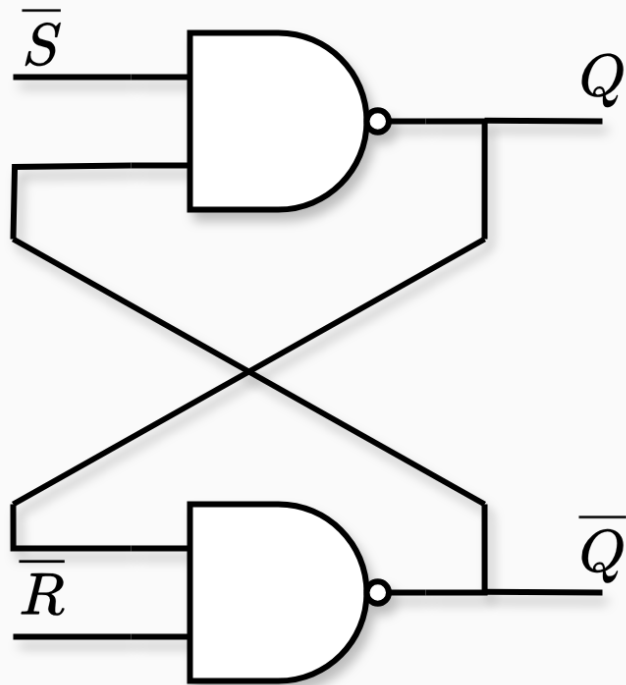
Latch SR. Metaestabilidad



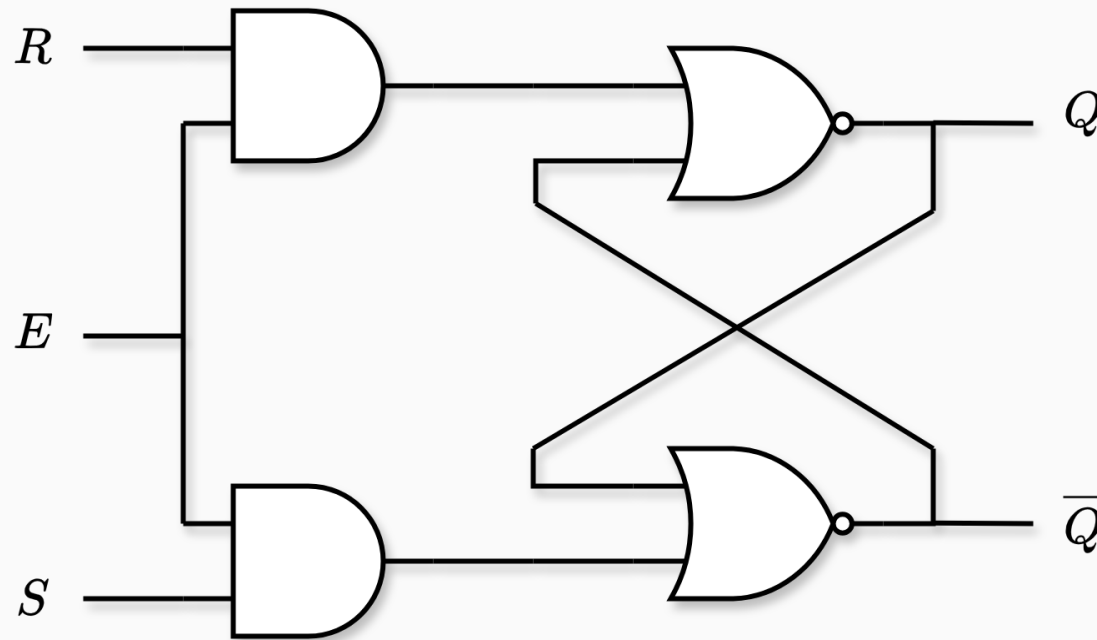
Latch SR. Metaestabilidad



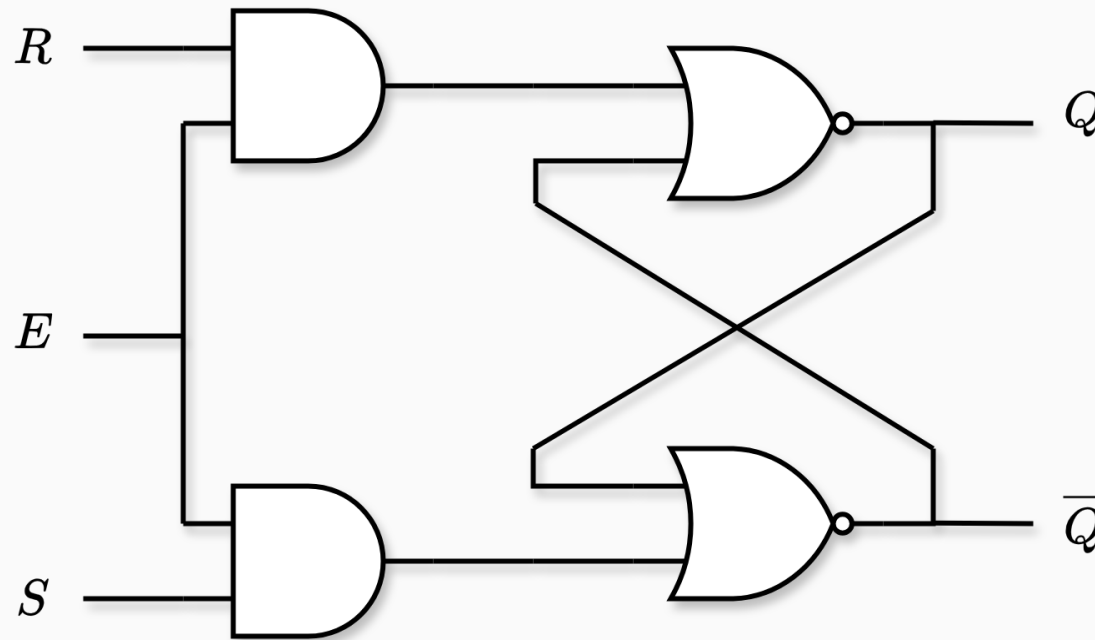
Latch SR. Construcción con NANDs



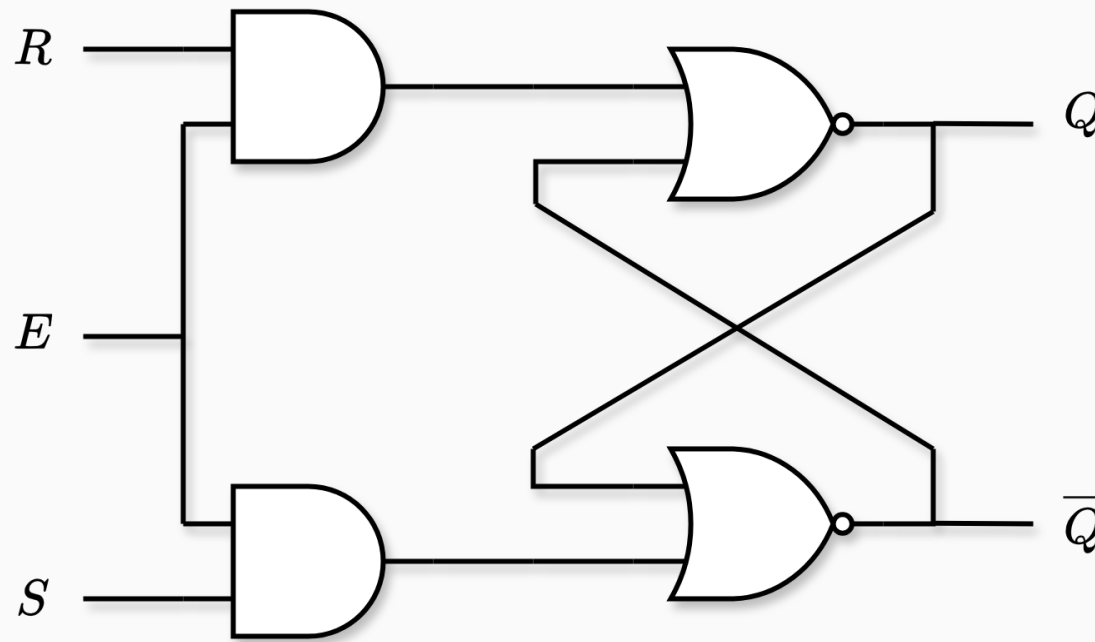
$\overline{S}$	$\overline{R}$	Q	$\overline{Q}$
1	1	latch	latch
1	0	0	1
0	1	1	0
0	0	U	U



- El comportamiento es el mismo que el latch SR, pero solo se activa cuando E es 1.



- El comportamiento es el mismo que el latch SR, pero solo se activa cuando E es 1.
- Si la señal E es 0, entonces el latch se mantiene en su estado actual.



- El comportamiento es el mismo que el latch SR, pero solo se activa cuando E es 1.
- Si la señal E es 0, entonces el latch se mantiene en su estado actual.
- Si la señal E es un clock, entonces el latch se dice que es *level-sensitive*.

Latch JK

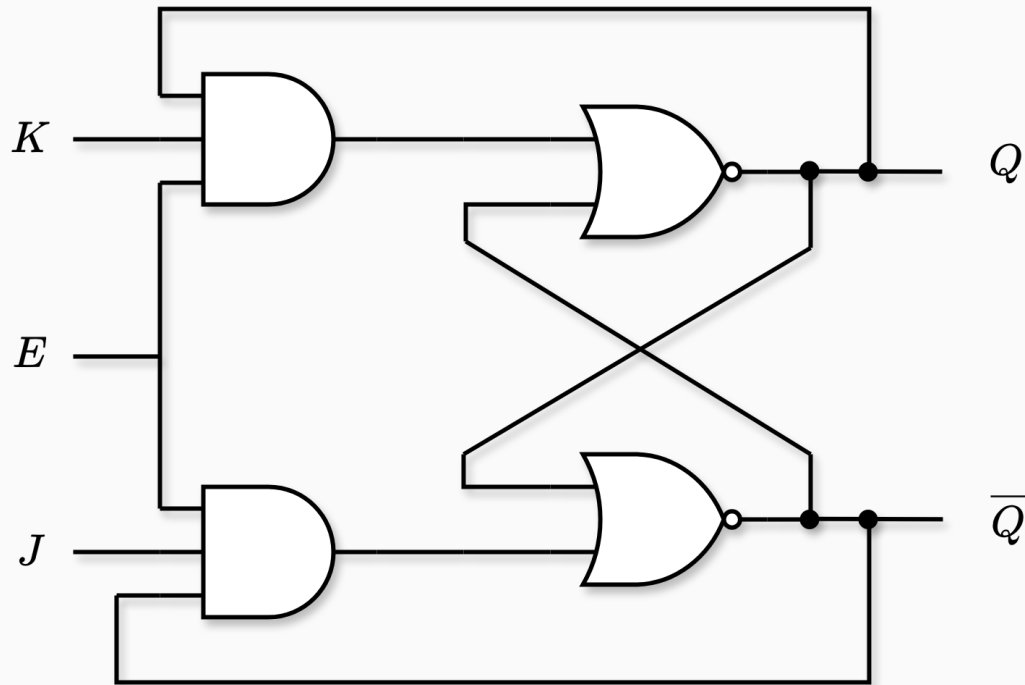


Tabla de verdad:

E	J	K	Q	$\overline{Q}$
1	1	0		
1	0	1		
1	0	0		
1	1	1		
0	–	–		

Latch JK

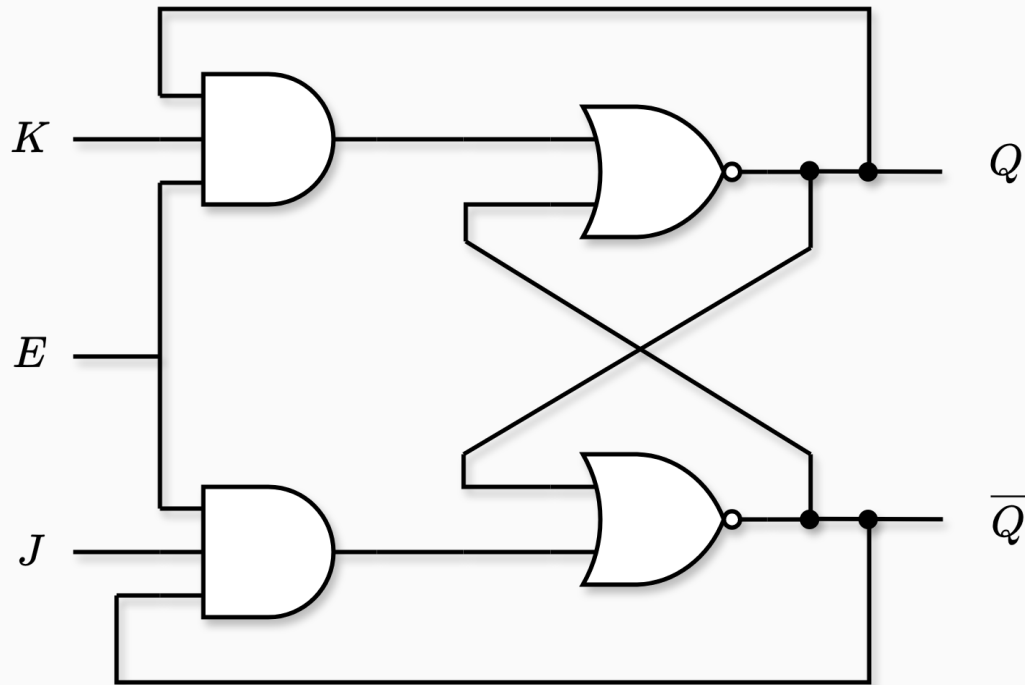


Tabla de verdad:

E	J	K	Q	$\overline{Q}$
1	1	0	1	0
1	0	1		
1	0	0		
1	1	1		
0	–	–		

Latch JK

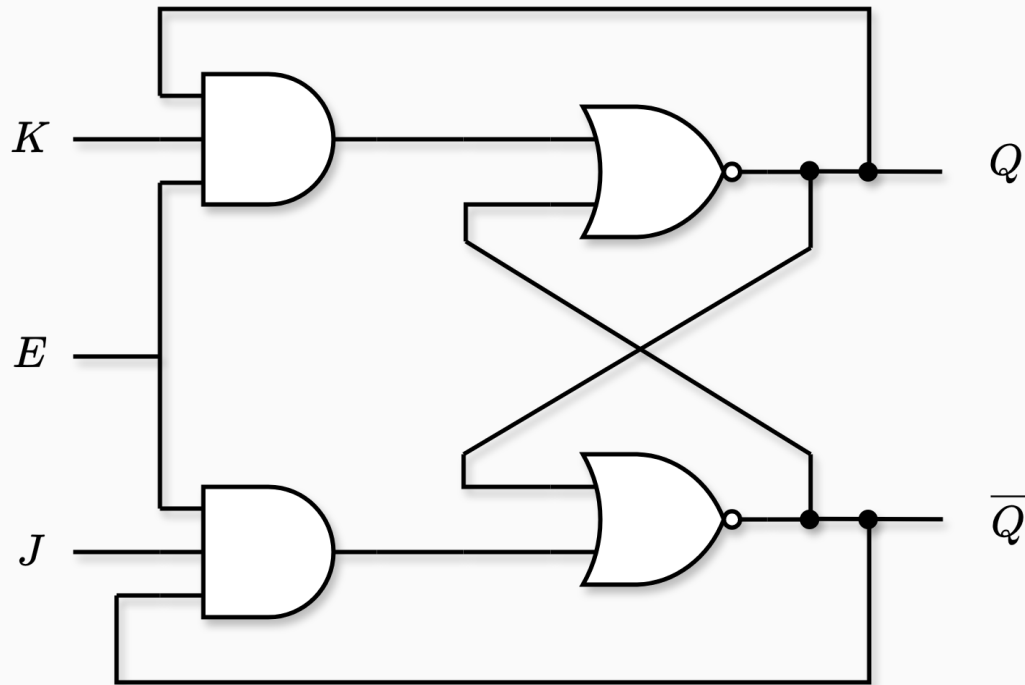


Tabla de verdad:

E	J	K	Q	$\overline{Q}$
1	1	0	1	0
1	0	1	0	1
1	0	0		
1	1	1		
0	–	–		

Latch JK

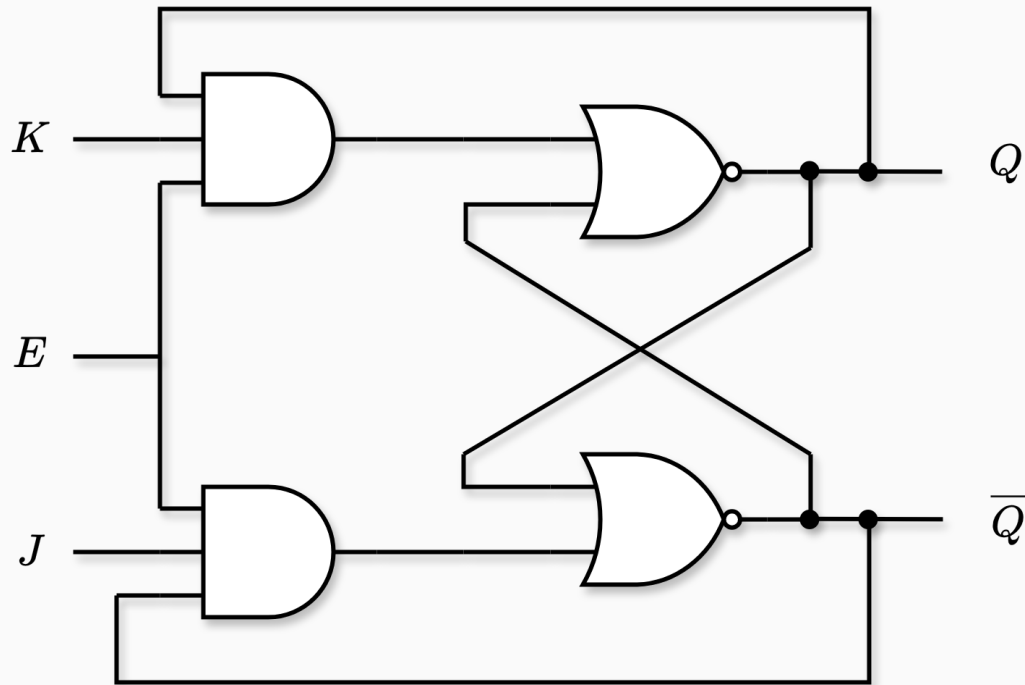


Tabla de verdad:

E	J	K	Q	$\overline{Q}$
1	1	0	1	0
1	0	1	0	1
1	0	0	Q_{prev}	$\overline{Q}_{\text{prev}}$
1	1	1		
0	–	–		

Latch JK

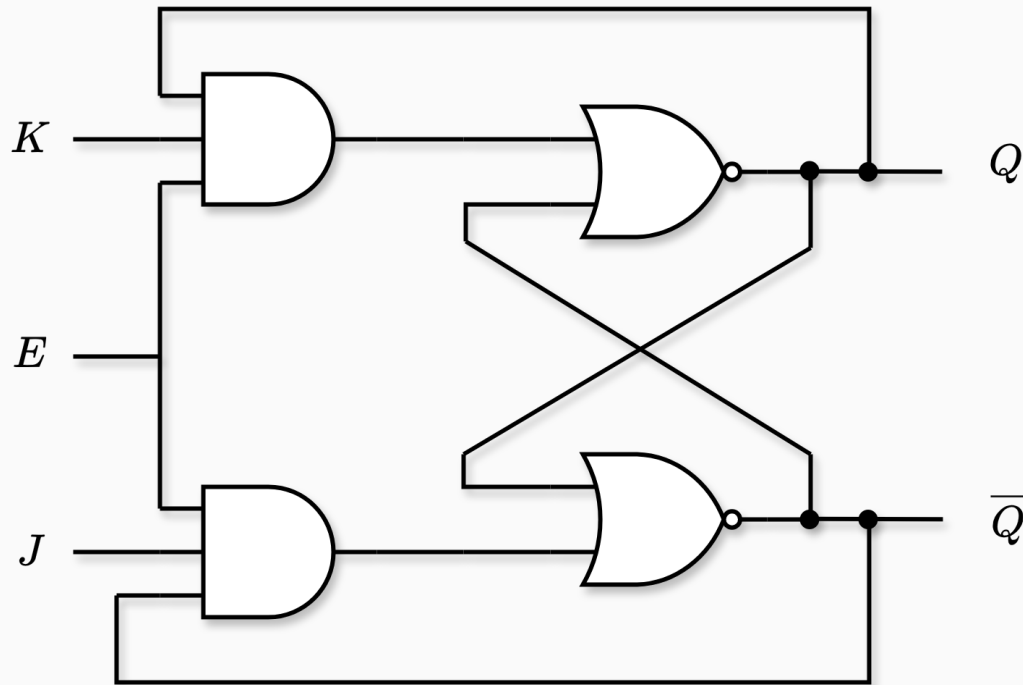


Tabla de verdad:

E	J	K	Q	$\overline{Q}$
1	1	0	1	0
1	0	1	0	1
1	0	0	Q_{prev}	$\overline{Q}_{\text{prev}}$
1	1	1	$\overline{Q}_{\text{prev}}$	Q_{prev}
0	—	—		

Latch JK

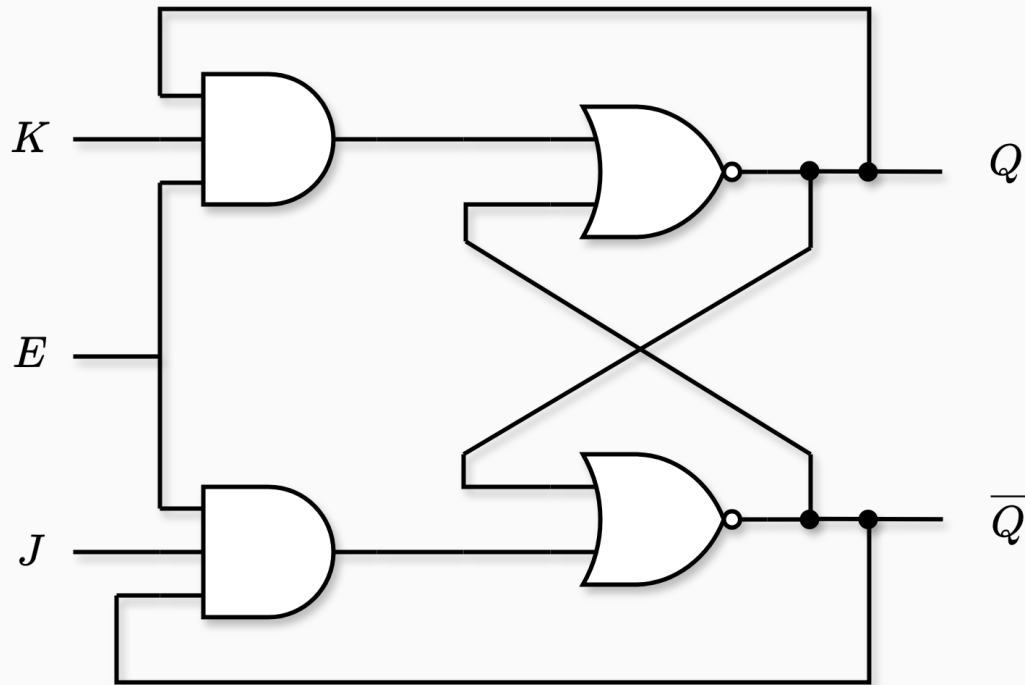


Tabla de verdad:

E	J	K	Q	$\overline{Q}$
1	1	0	1	0
1	0	1	0	1
1	0	0	Q_{prev}	$\overline{Q}_{\text{prev}}$
1	1	1	$\overline{Q}_{\text{prev}}$	Q_{prev}
0	—	—	Q_{prev}	$\overline{Q}_{\text{prev}}$

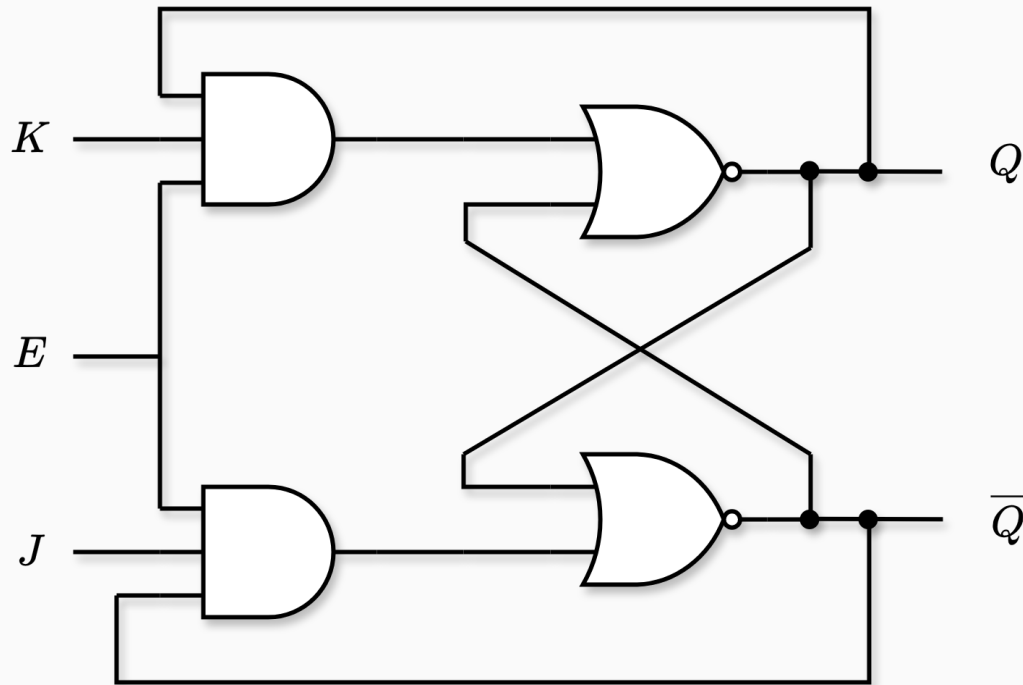
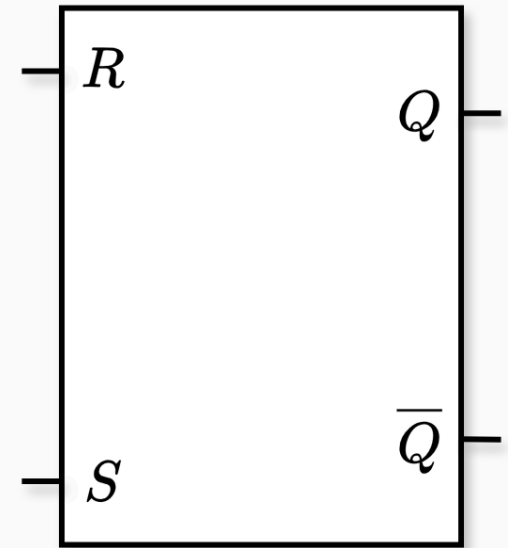
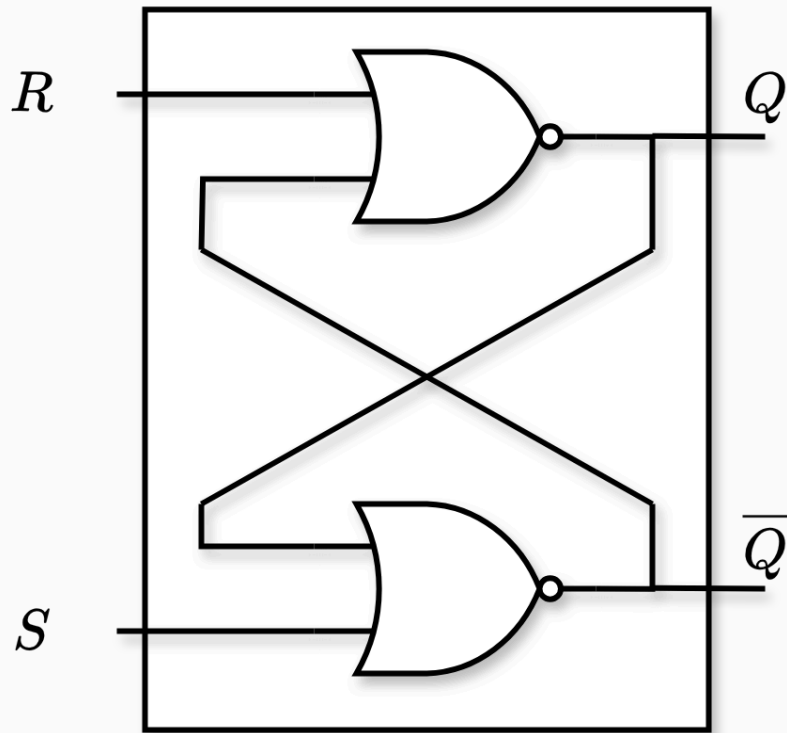


Tabla de verdad:

E	J	K	Q	$\overline{Q}$
1	1	0	1	0
1	0	1	0	1
1	0	0	Q_{prev}	$\overline{Q}_{\text{prev}}$
1	1	1	$\overline{Q}_{\text{prev}}$	Q_{prev}
0	–	–	Q_{prev}	$\overline{Q}_{\text{prev}}$

Con $J, K = (1, 1)$:

- El valor de las salidas está ahora definido.
- El circuito oscila (estado inestable).



Latch D

- Nos permite almacenar 1 bit
- Tiene una entrada de **datos** y una de **control**

Latch D:

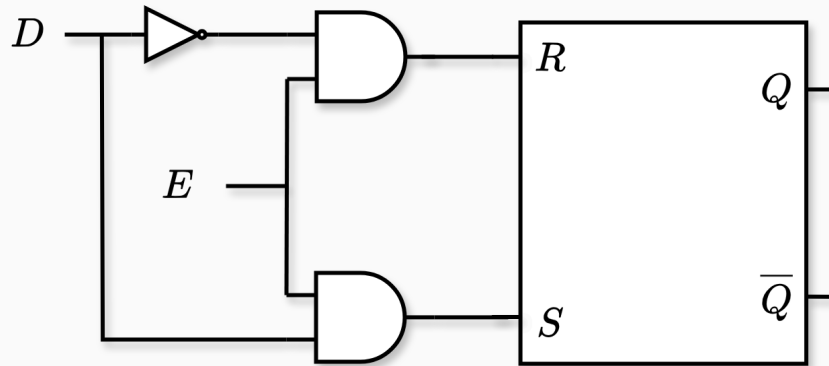


Tabla de verdad:

E	D	Q	$\overline{Q}$
0	–		
1	0		
1	1		

Latch D

- Nos permite almacenar 1 bit
- Tiene una entrada de **datos** y una de **control**

Latch D:

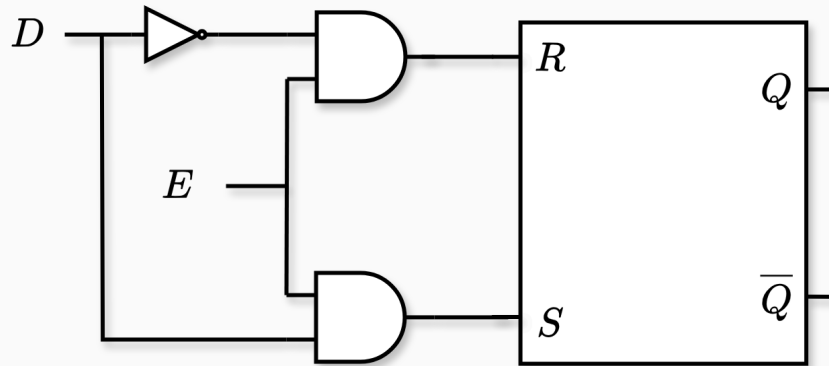


Tabla de verdad:

E	D	Q	$\overline{Q}$
0	–	Q_{prev}	$\overline{Q}_{\text{prev}}$
1	0		
1	1		

- Nos permite almacenar 1 bit
- Tiene una entrada de **datos** y una de **control**

Latch D:

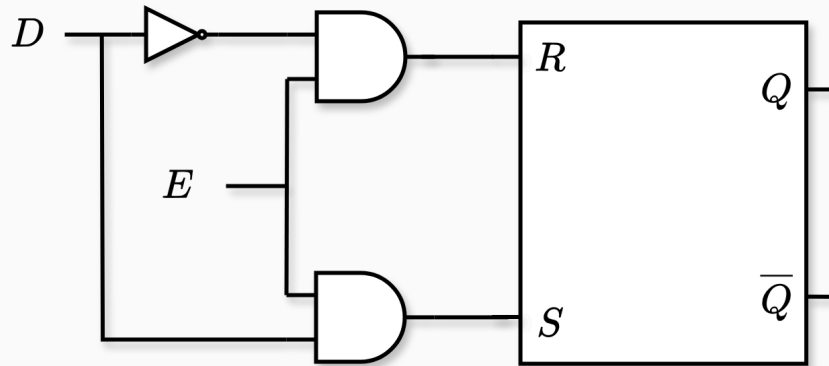


Tabla de verdad:

E	D	Q	$\bar{Q}$
0	–	Q_{prev}	$\bar{Q}_{\text{prev}}$
1	0	0	1
1	1		

Latch D

- Nos permite almacenar 1 bit
- Tiene una entrada de **datos** y una de **control**

Latch D:

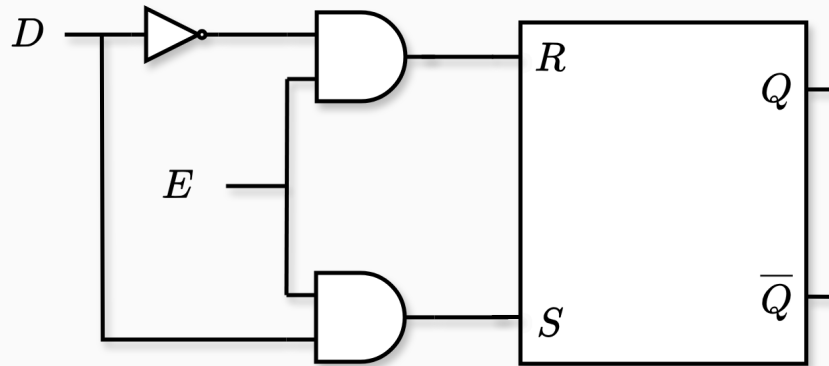


Tabla de verdad:

E	D	Q	$\bar{Q}$
0	–	Q_{prev}	$\bar{Q}_{\text{prev}}$
1	0	0	1
1	1	1	0

Latch D

- Nos permite almacenar 1 bit
- Tiene una entrada de **datos** y una de **control**

Latch D:

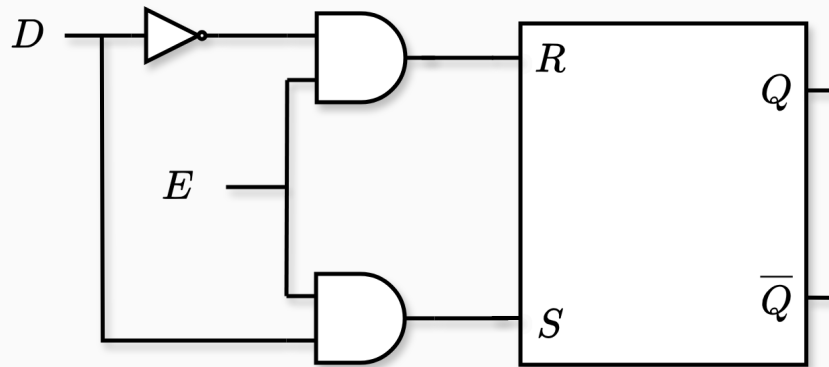
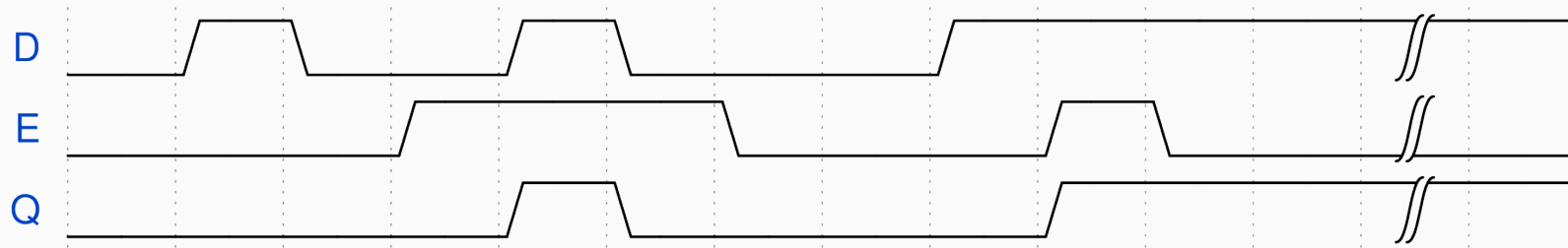


Tabla de verdad:

E	D	Q	$\overline{Q}$
0	–	Q_{prev}	$\overline{Q}_{\text{prev}}$
1	0	0	1
1	1	1	0



- Nos permite almacenar 1 bit
- Tiene una entrada de **datos** y una de **control**

Latch D:

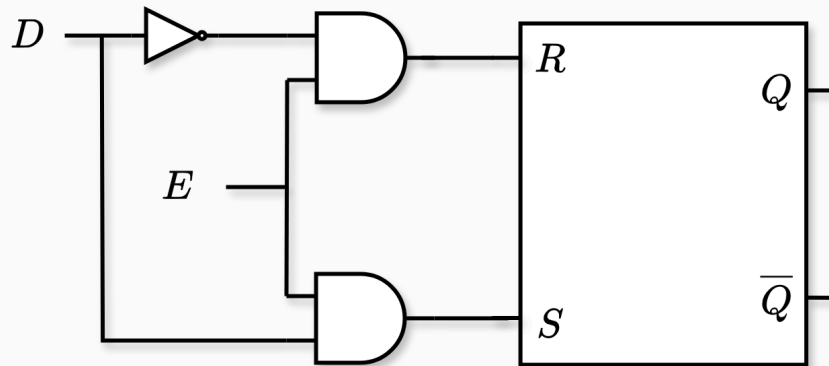
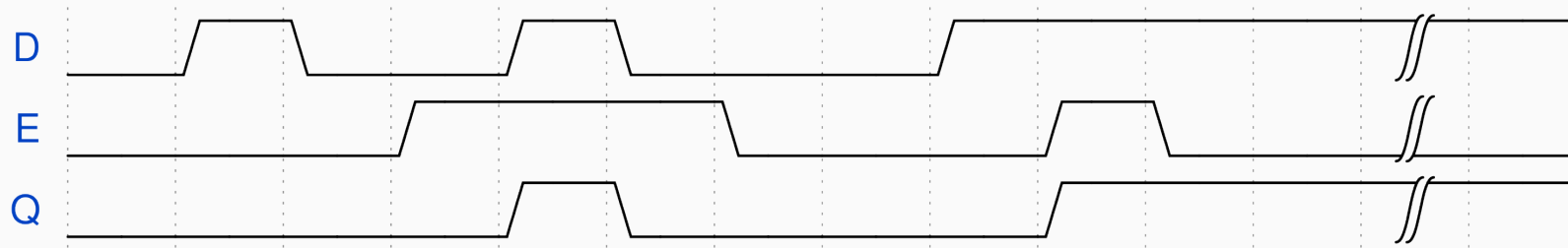


Tabla de verdad:

E	D	Q	$\bar{Q}$
0	–	Q_{prev}	$\bar{Q}_{\text{prev}}$
1	0	0	1
1	1	1	0



- D no debe cambiar durante una ventana de tiempo alrededor del flanco de bajada de E

- Nos permite almacenar 1 bit
- Tiene una entrada de **datos** y una de **control**

Latch D:

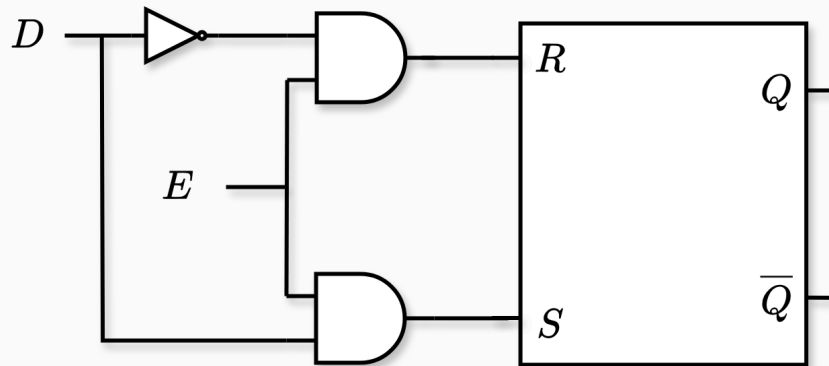
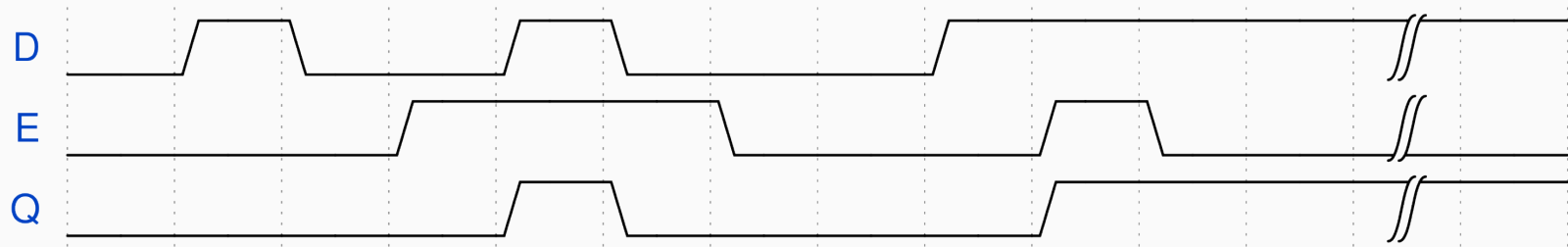


Tabla de verdad:

E	D	Q	$\bar{Q}$
0	–	Q_{prev}	$\bar{Q}_{\text{prev}}$
1	0	0	1
1	1	1	0



- D no debe cambiar durante una ventana de tiempo alrededor del flanco de bajada de E
- Los tiempos no se pueden predecir (dependen de D)

- Nos permite almacenar 1 bit
- Tiene una entrada de **datos** y una de **control**

Latch D:

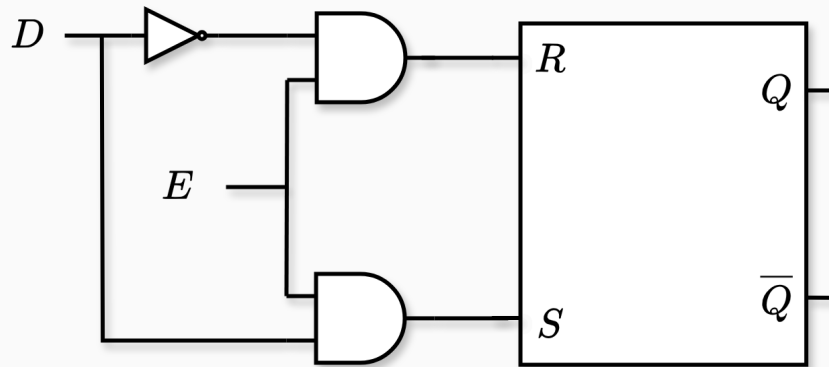
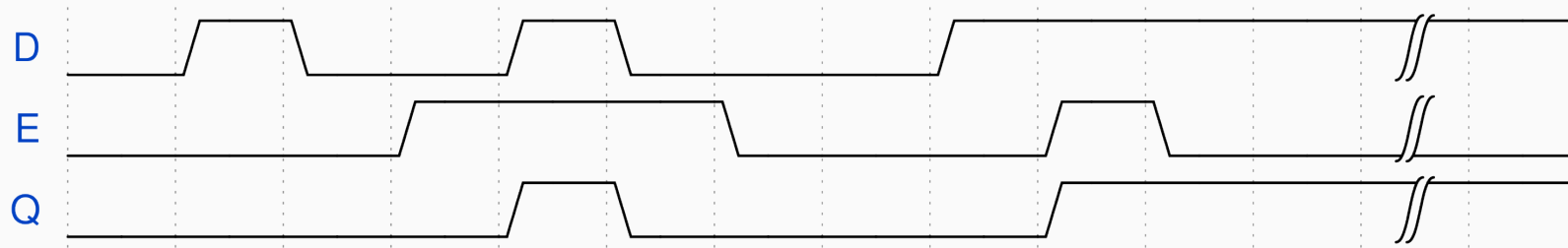


Tabla de verdad:

E	D	Q	$\bar{Q}$
0	–	Q_{prev}	$\bar{Q}_{\text{prev}}$
1	0	0	1
1	1	1	0



- D no debe cambiar durante una ventana de tiempo alrededor del flanco de bajada de E
- Los tiempos no se pueden predecir (dependen de D)
- Puede causar carreras si existe un lazo en el circuito externo.

Flip-Flops



Sincronismo

En un sistema de cómputo necesitamos *sincronizar* los cambios de estado para que sucedan en el mismo momento y a intervalos regulares

Para eso necesitamos:

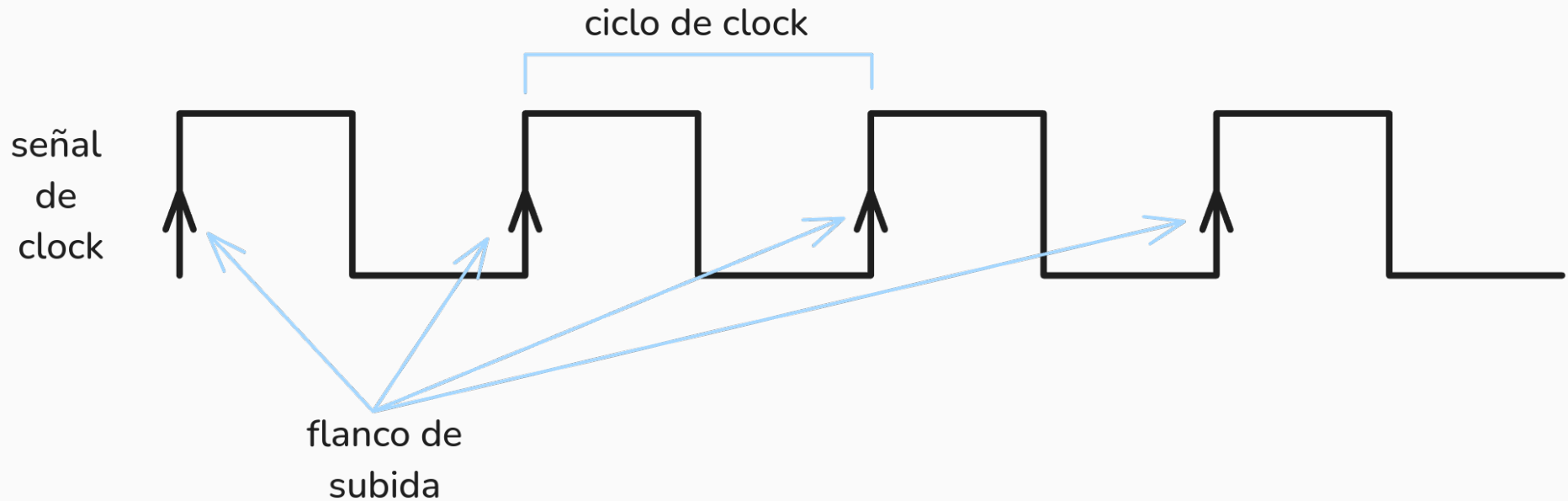
- Una señal de sincronismo (**clock**)

Sincronismo

En un sistema de cómputo necesitamos *sincronizar* los cambios de estado para que sucedan en el mismo momento y a intervalos regulares

Para eso necesitamos:

- Una señal de sincronismo (**clock**)
- Un dispositivo de memoria que no sea level-sensitive (**flip-flop**)



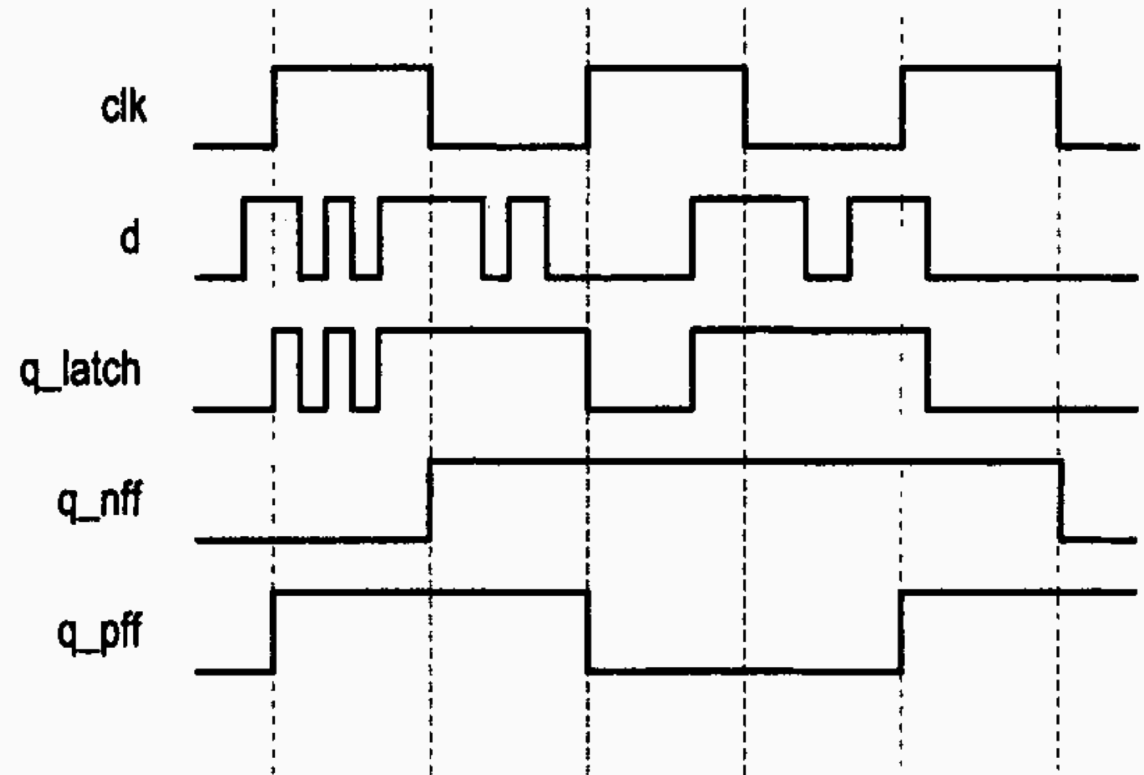
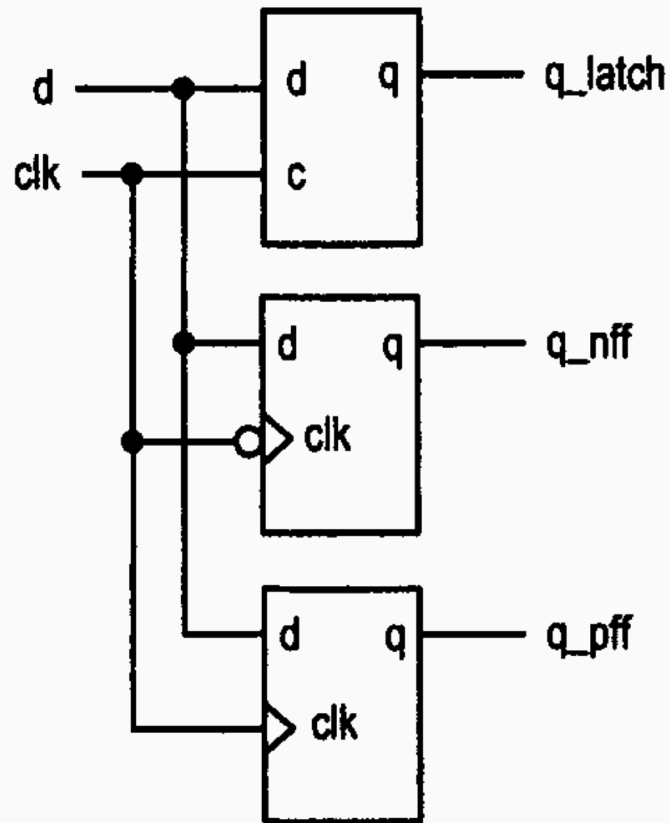
Período T : tiempo que tarda en completar un ciclo.

Frecuencia f : cantidad de ciclos por unidad de tiempo.

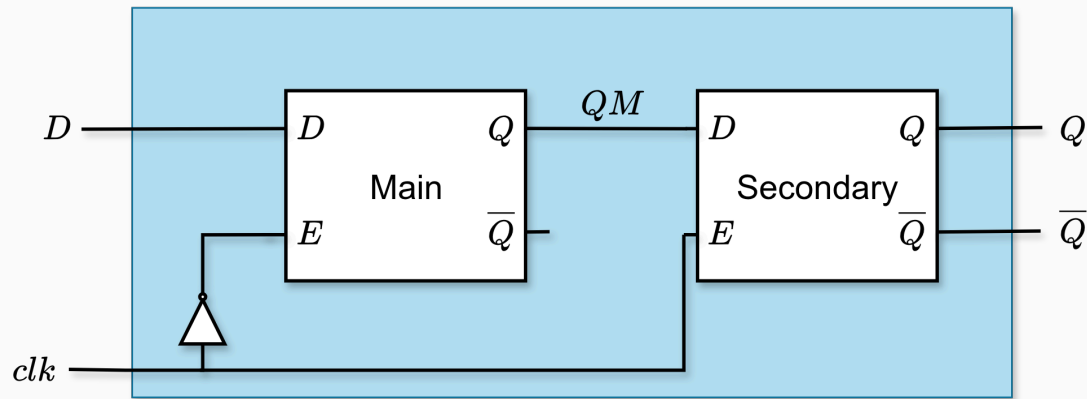
Son recíprocos:

$$f = \frac{1}{T}$$

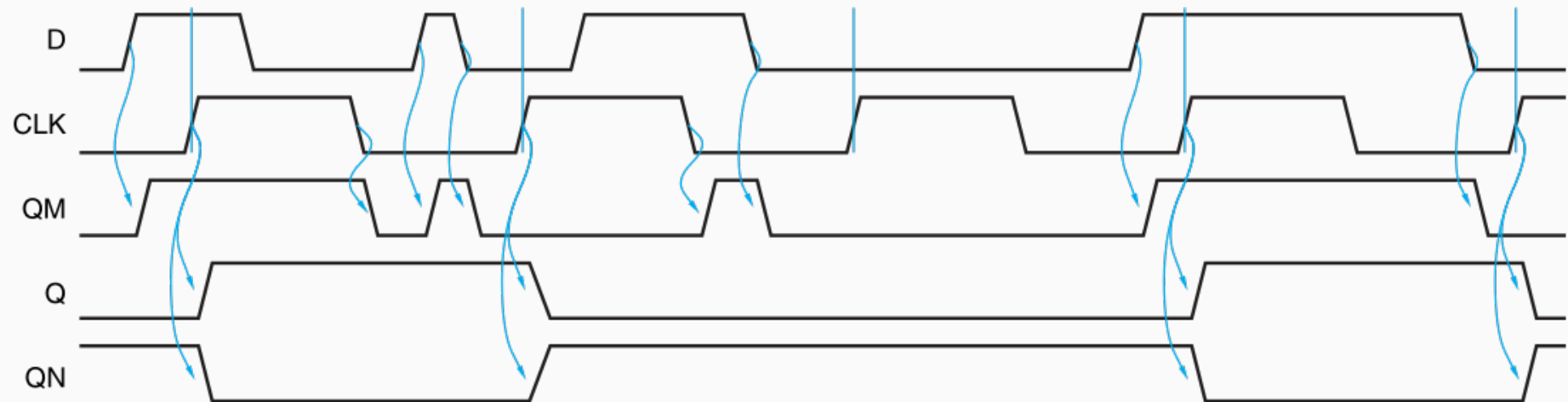
Sincronizando...



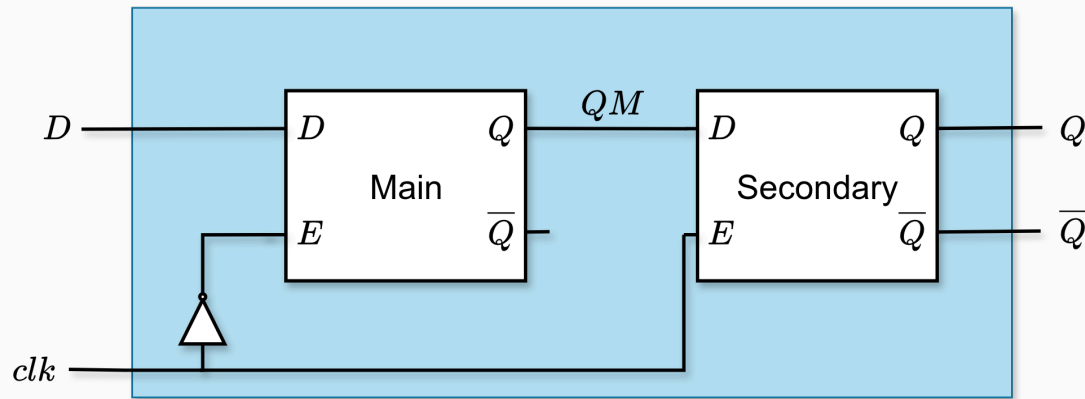
Flip-Flop D



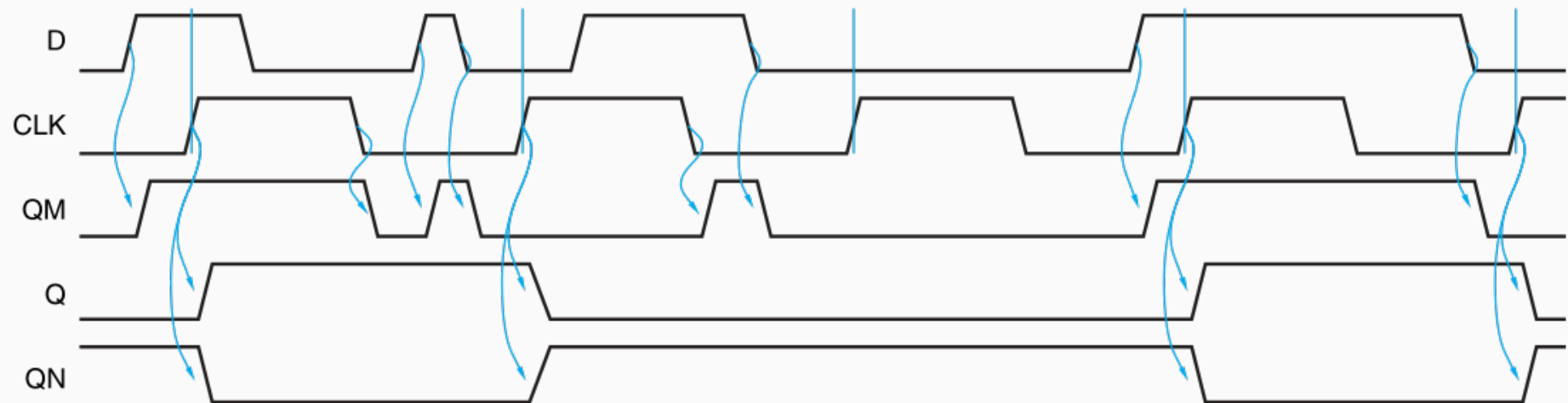
D	clock	Q	$\bar{Q}$
0			
1			
—	0		
—	1		



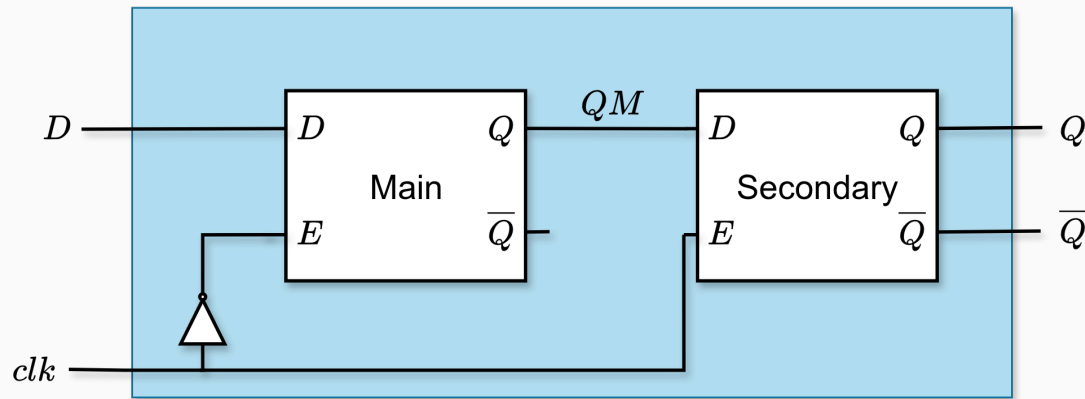
Flip-Flop D



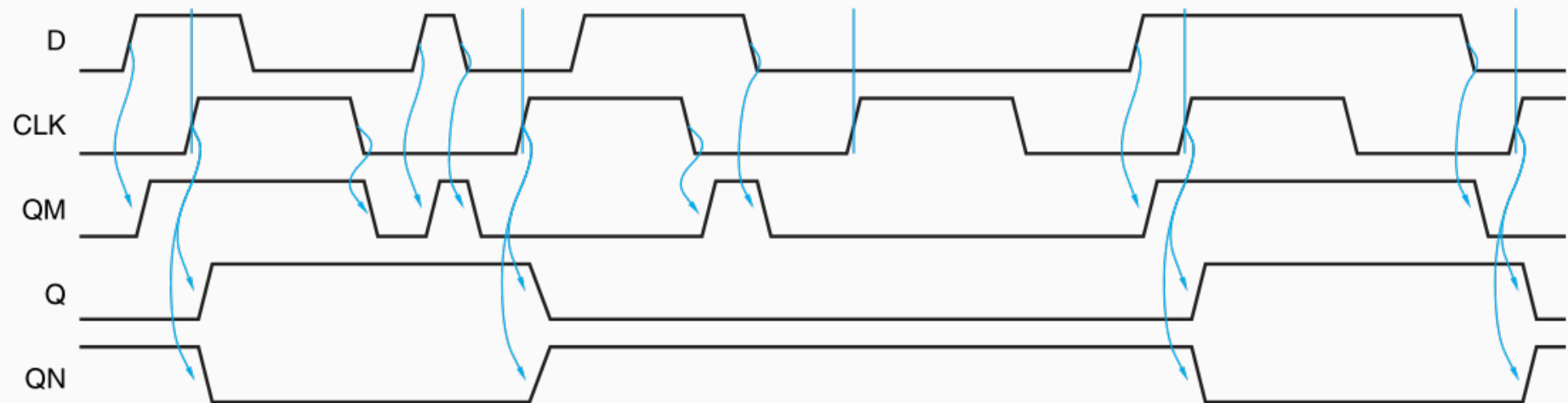
D	clock	Q	$\bar{Q}$
0		0	1
1		1	0
—	0	—	—
—	1	—	—



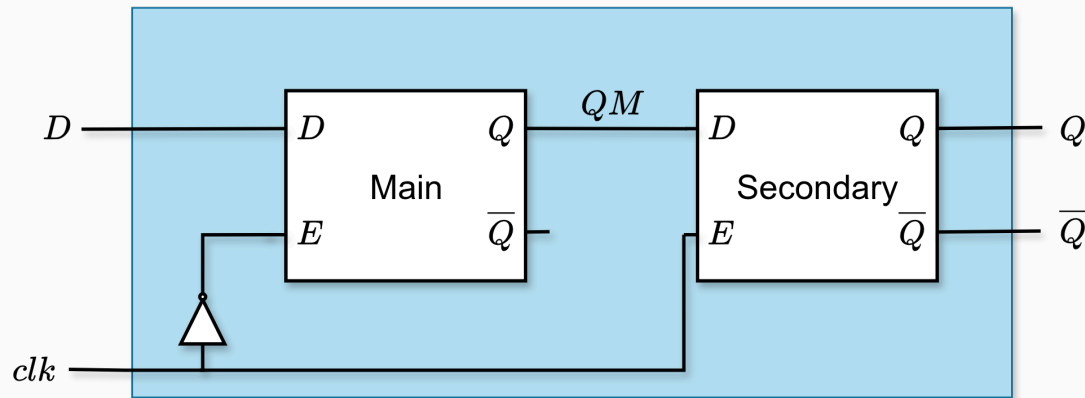
Flip-Flop D



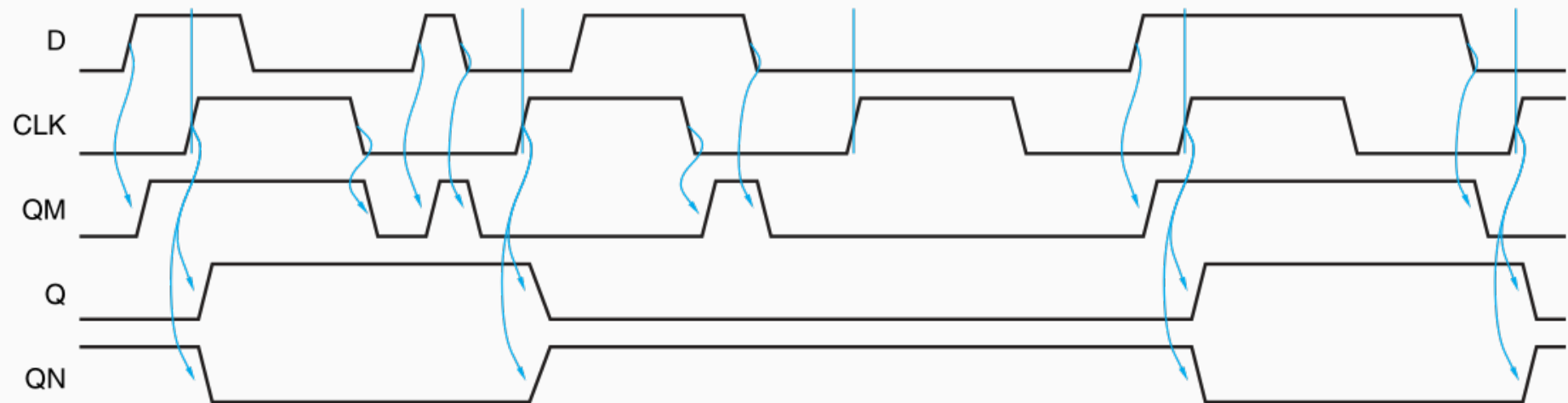
D	clock	Q	$\bar{Q}$
0		0	1
1		1	0
—	0		
—	1		



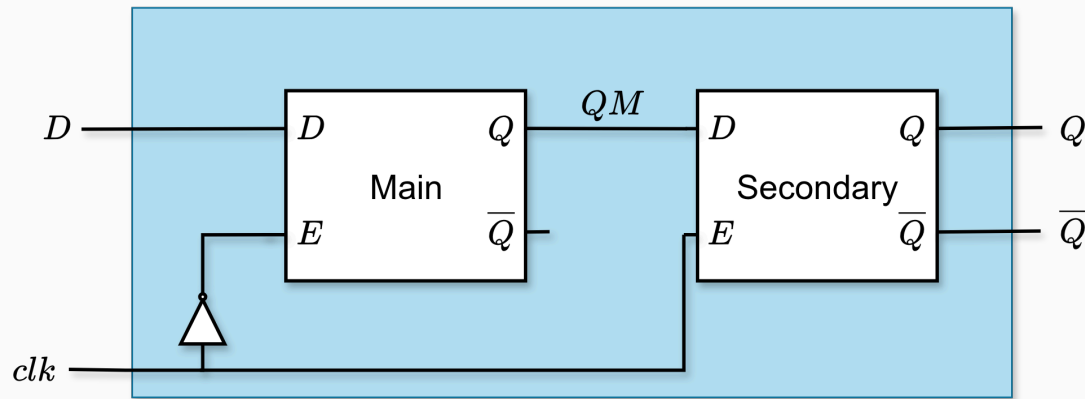
Flip-Flop D



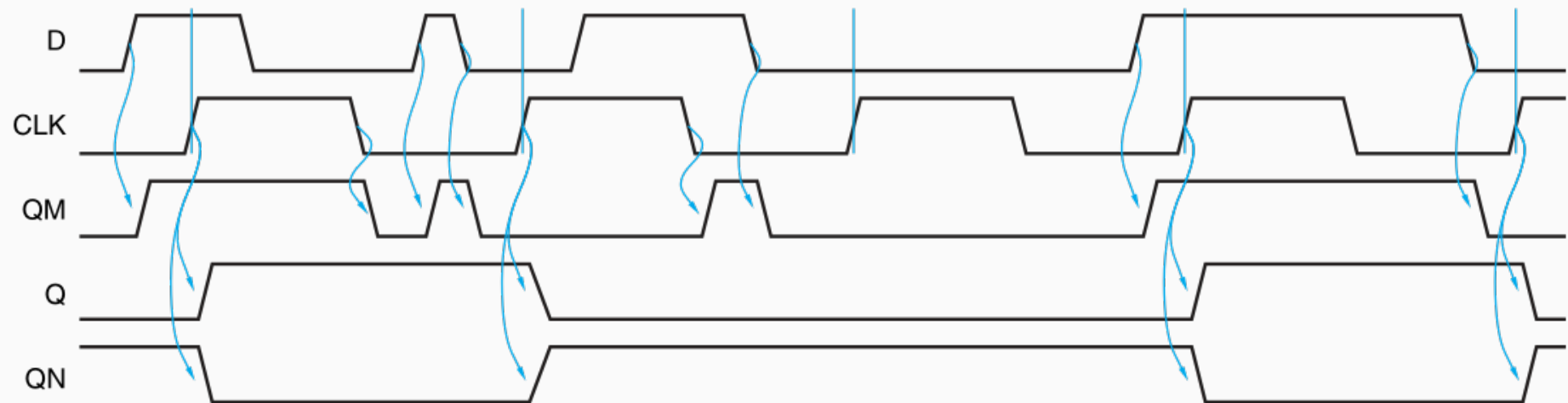
D	clock	Q	$\overline{Q}$
0		0	1
1		1	0
—	0	Q_{prev}	$\overline{Q}_{\text{prev}}$
—	1		



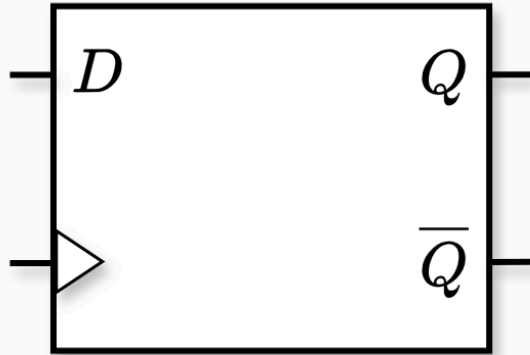
Flip-Flop D



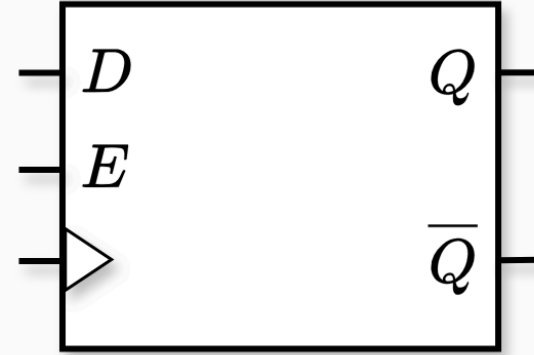
D	clock	Q	$\bar{Q}$
0		0	1
1		1	0
—	0	Q_{prev}	$\bar{Q}_{\text{prev}}$
—	1	Q_{prev}	$\bar{Q}_{\text{prev}}$



Flip-Flop D con Enable

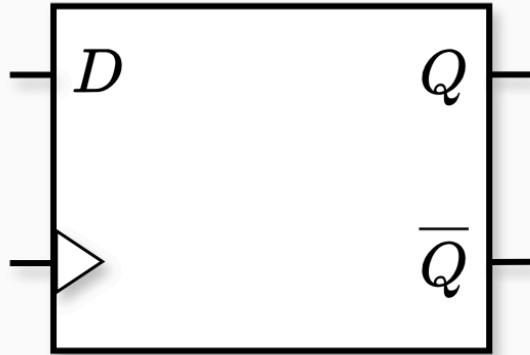


Flip-Flop D

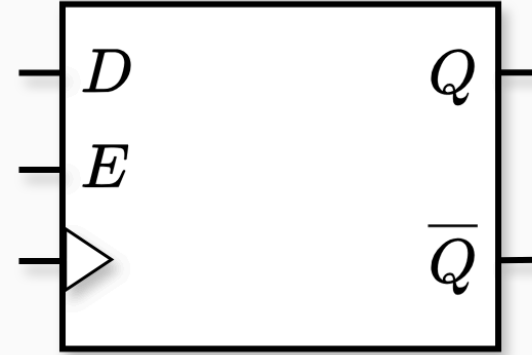


Flip-Flop D con Enable

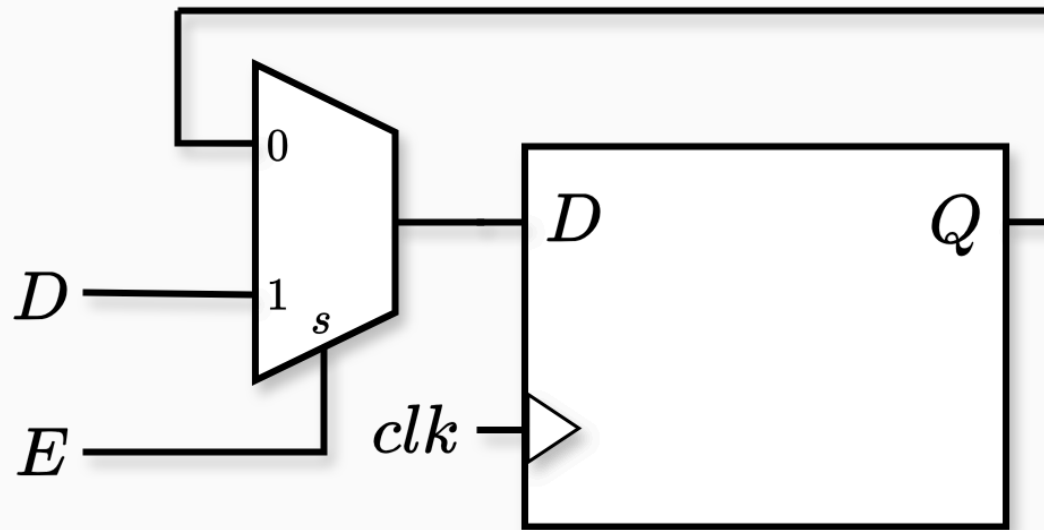
Flip-Flop D con Enable

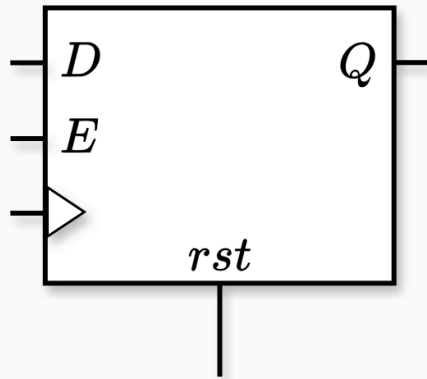


Flip-Flop D

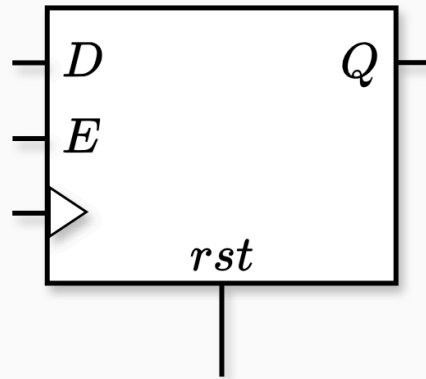


Flip-Flop D con Enable

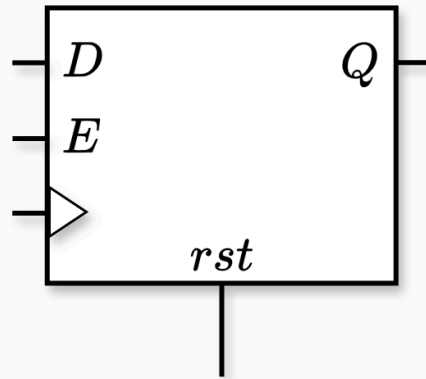




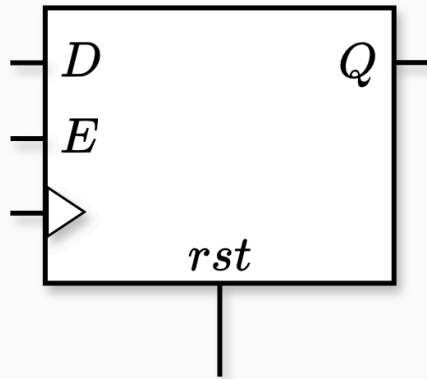
D	E	rst_a	rst_s	clk	Q
0	1	0	0		
1	1	0	0		
–	0	0	0		
–	–	0	1		
–	–	1	–	–	



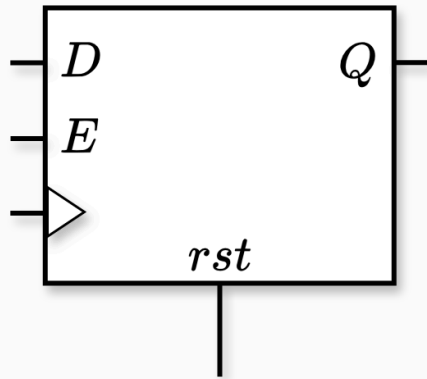
D	E	rst_a	rst_s	clk	Q
0	1	0	0		0
1	1	0	0		
–	0	0	0		
–	–	0	1		
–	–	1	–	–	



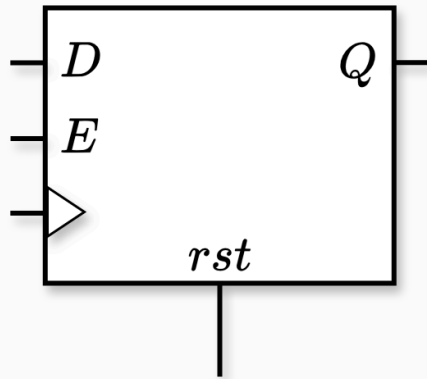
D	E	rst_a	rst_s	clk	Q
0	1	0	0		0
1	1	0	0		1
–	0	0	0		
–	–	0	1		
–	–	1	–	–	



D	E	rst_a	rst_s	clk	Q
0	1	0	0		0
1	1	0	0		1
–	0	0	0		Q_{prev}
–	–	0	1		
–	–	1	–	–	

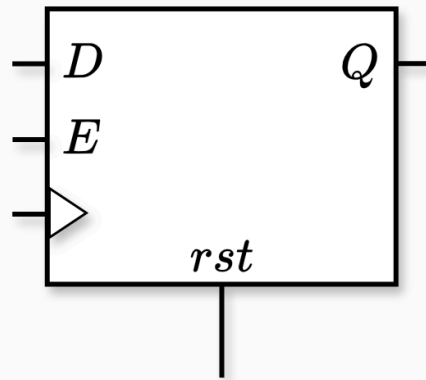


D	E	rst_a	rst_s	clk	Q
0	1	0	0		0
1	1	0	0		1
–	0	0	0		Q_{prev}
–	–	0	1		0
–	–	1	–	–	

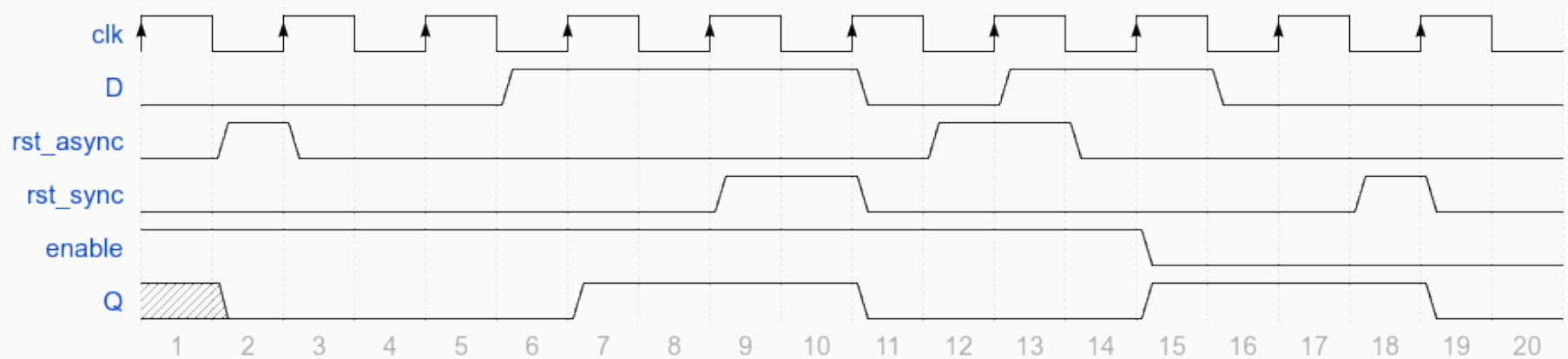


D	E	rst_a	rst_s	clk	Q
0	1	0	0		0
1	1	0	0		1
–	0	0	0		Q_{prev}
–	–	0	1		0
–	–	1	–	–	0

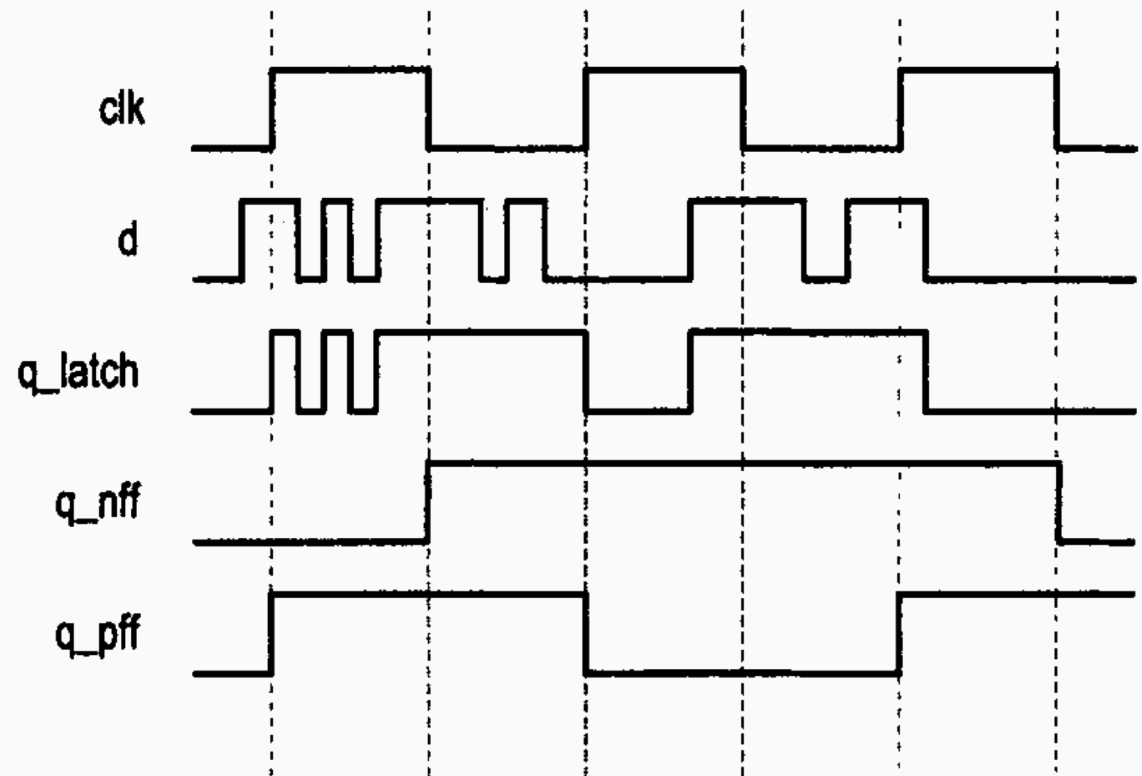
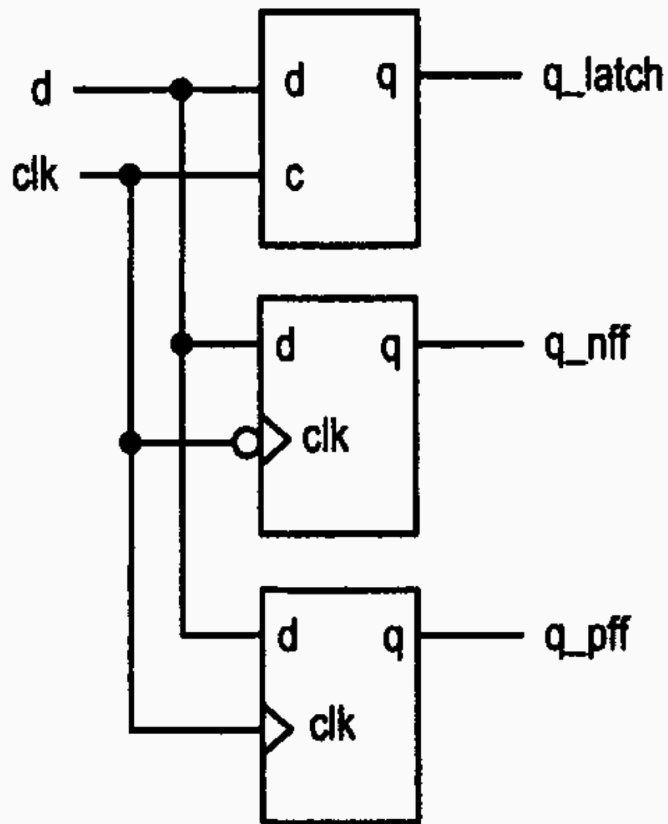
Reset



D	E	rst_a	rst_s	clk	Q
0	1	0	0		0
1	1	0	0		1
–	0	0	0		Q_{prev}
–	–	0	1		0
–	–	1	–	–	0



Ahora podemos entender bien las diferencias:



Volviendo al latch J-K, ahora con detección de flanco podemos obtener un comportamiento más *adecuado*:

Ahora lo podemos representar como:

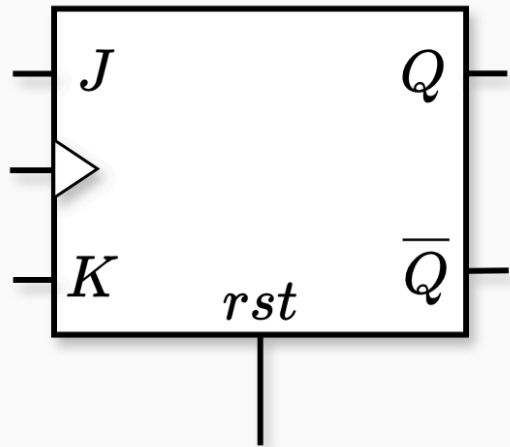


Tabla de verdad:

J	K	clk	Q	$\overline{Q}$
1	0	1 ↗		
0	1	1 ↗		
0	0	1 ↗		
1	1	1 ↗		
–	–	0		

Volviendo al latch J-K, ahora con detección de flanco podemos obtener un comportamiento más *adecuado*:

Ahora lo podemos representar como:

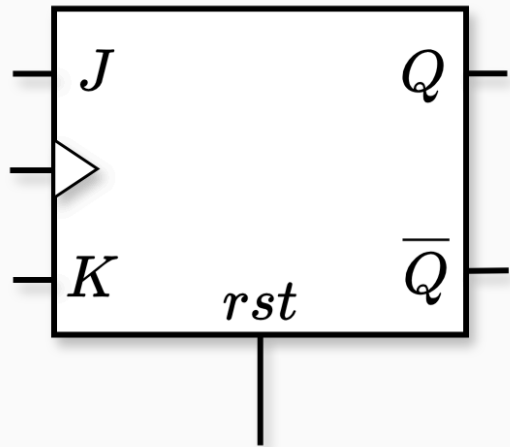


Tabla de verdad:

J	K	clk	Q	$\overline{Q}$
1	0	1 ↗	1	0
0	1	1 ↗		
0	0	1 ↗		
1	1	1 ↗		
–	–	0		

Volviendo al latch J-K, ahora con detección de flanco podemos obtener un comportamiento más *adecuado*:

Ahora lo podemos representar como:

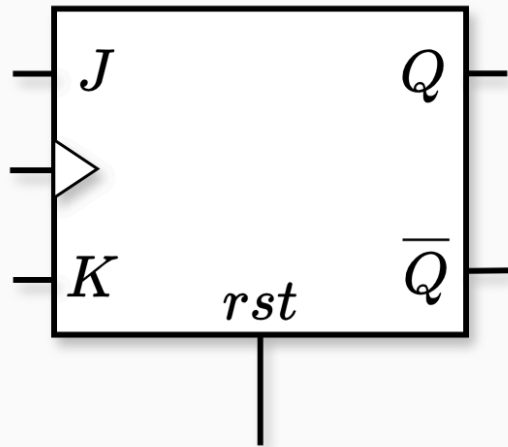


Tabla de verdad:

J	K	clk	Q	$\overline{Q}$
1	0	1 ↗	1	0
0	1	1 ↗	0	1
0	0	1 ↗		
1	1	1 ↗		
–	–	0		

Volviendo al latch J-K, ahora con detección de flanco podemos obtener un comportamiento más *adecuado*:

Ahora lo podemos representar como:

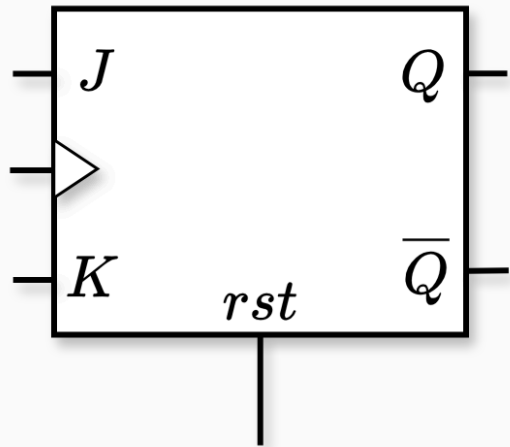


Tabla de verdad:

J	K	clk	Q	$\overline{Q}$
1	0	1 ↗	1	0
0	1	1 ↗	0	1
0	0	1 ↗	Q_{prev}	$\overline{Q}_{\text{prev}}$
1	1	1 ↗		
–	–	0		

Volviendo al latch J-K, ahora con detección de flanco podemos obtener un comportamiento más *adecuado*:

Ahora lo podemos representar como:

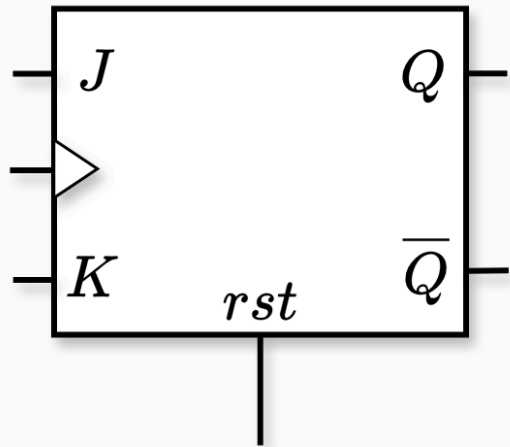


Tabla de verdad:

J	K	clk	Q	$\overline{Q}$
1	0	1 ↗	1	0
0	1	1 ↗	0	1
0	0	1 ↗	Q_{prev}	$\overline{Q}_{\text{prev}}$
1	1	1 ↗	$\overline{Q}_{\text{prev}}$	Q_{prev}
–	–	0		

Volviendo al latch J-K, ahora con detección de flanco podemos obtener un comportamiento más *adecuado*:

Ahora lo podemos representar como:

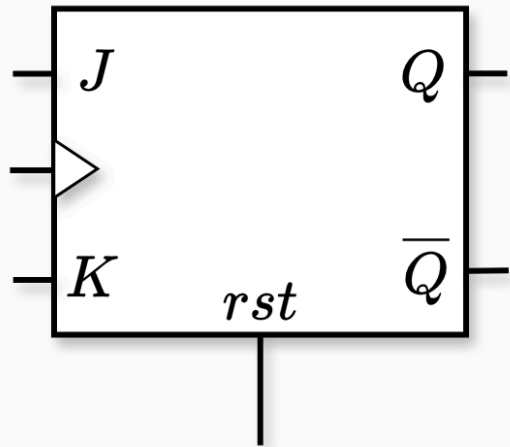


Tabla de verdad:

J	K	clk	Q	$\overline{Q}$
1	0	1 ↗	1	0
0	1	1 ↗	0	1
0	0	1 ↗	Q_{prev}	$\overline{Q}_{\text{prev}}$
1	1	1 ↗	$\overline{Q}_{\text{prev}}$	Q_{prev}
—	—	0	Q_{prev}	Q_{prev}

Volviendo al latch J-K, ahora con detección de flanco podemos obtener un comportamiento más *adecuado*:

Ahora lo podemos representar como:

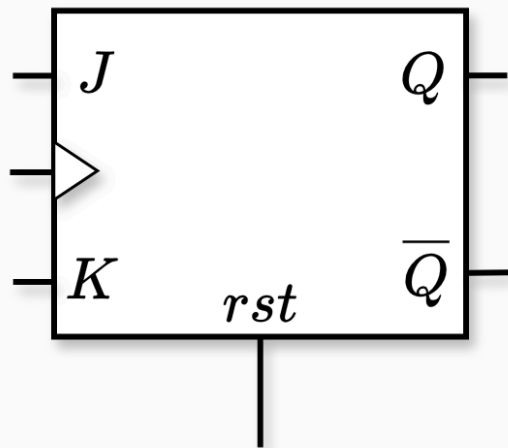


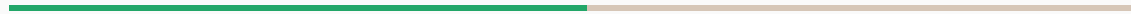
Tabla de verdad:

J	K	clk	Q	$\overline{Q}$
1	0	1 ↗	1	0
0	1	1 ↗	0	1
0	0	1 ↗	Q_{prev}	$\overline{Q}_{\text{prev}}$
1	1	1 ↗	$\overline{Q}_{\text{prev}}$	Q_{prev}
–	–	0	Q_{prev}	Q_{prev}

Ahora en el caso crítico donde $J, K = (1, 1)$ la salida tiene un estado y un tiempo de cambio bien definido:

Se niega el valor anterior cada 1 clock

Metaestabilidad

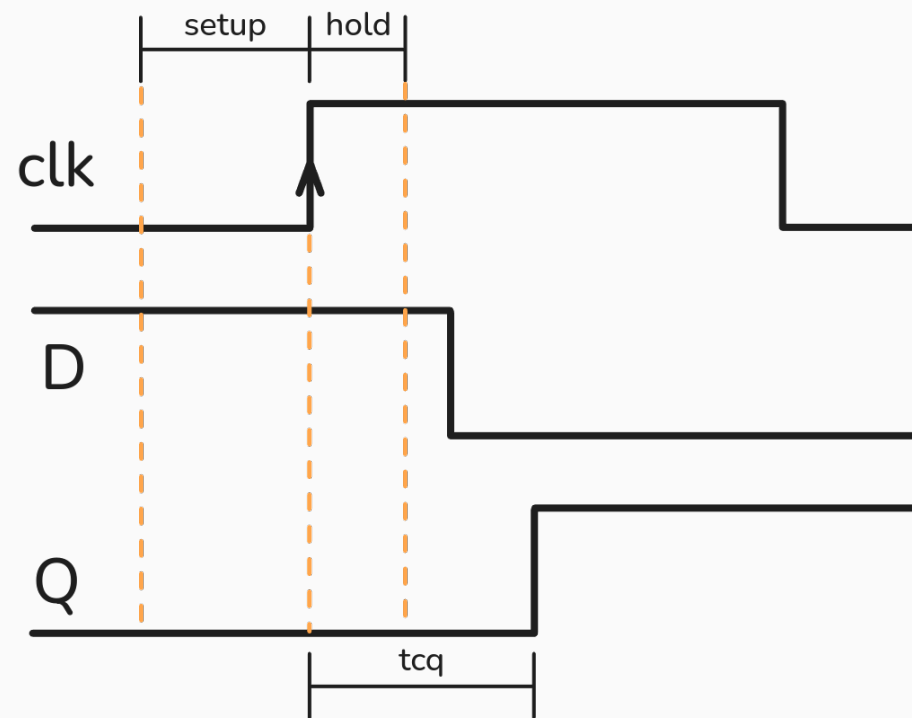


- En un flip-flop, la salida Q no cambia instantáneamente.

- En un flip-flop, la salida Q no cambia instantáneamente.
- No sólo eso, sino que la entrada debe permanecer estable durante una ventana de tiempo.

- En un flip-flop, la salida Q no cambia instantáneamente.
- No sólo eso, sino que la entrada debe permanecer estable durante una ventana de tiempo.
- Si la entrada cambia durante esa ventana, la salida puede quedar en un estado indefinido.

- En un flip-flop, la salida Q no cambia instantáneamente.
- No sólo eso, sino que la entrada debe permanecer estable durante una ventana de tiempo.
- Si la entrada cambia durante esa ventana, la salida puede quedar en un estado indefinido.



Transición adecuada

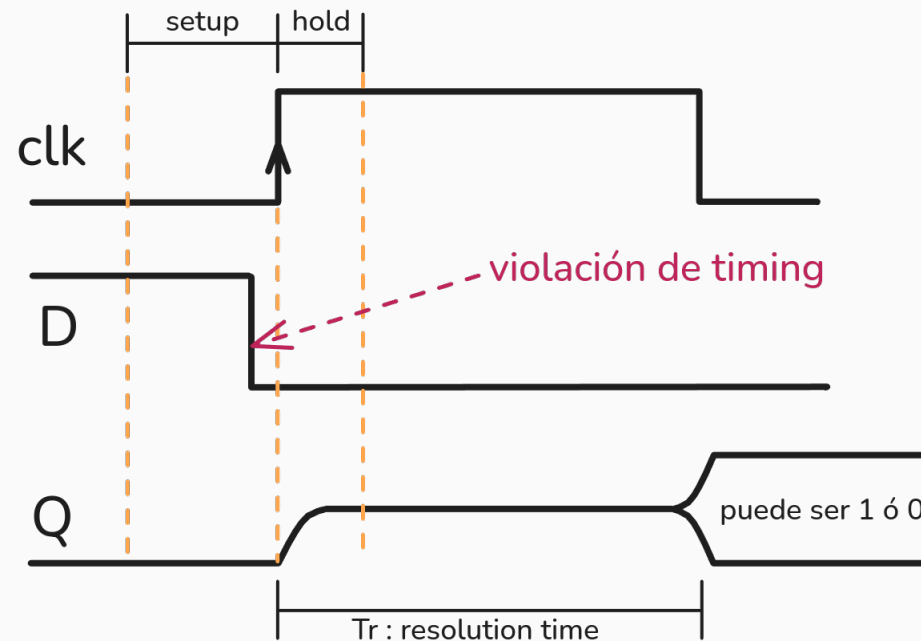
- Se dice que tenemos una *violación de tiempo* cuando la entrada cambia durante dicha ventana.

- Se dice que tenemos una *violación de tiempo* cuando la entrada cambia durante dicha ventana.
- La salida puede quedar en un estado **metaestable**.

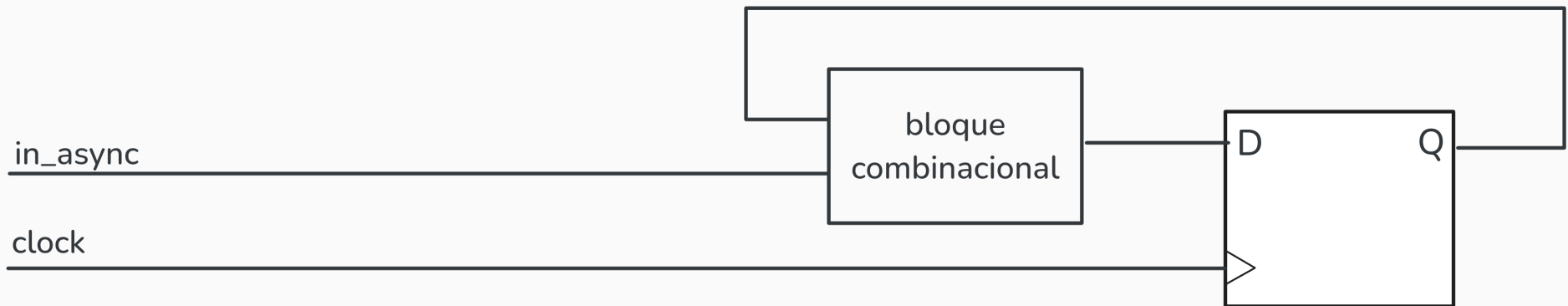
- Se dice que tenemos una *violación de tiempo* cuando la entrada cambia durante dicha ventana.
- La salida puede quedar en un estado **metaestable**.
- Eventualmente, la salida se estabilizará en un estado válido, luego de un *tiempo de resolución* T_r .

- Se dice que tenemos una *violación de tiempo* cuando la entrada cambia durante dicha ventana.
- La salida puede quedar en un estado **metaestable**.
- Eventualmente, la salida se estabilizará en un estado válido, luego de un *tiempo de resolución* T_r .
- T_r es una variable aleatoria, y no podemos predecir su valor.

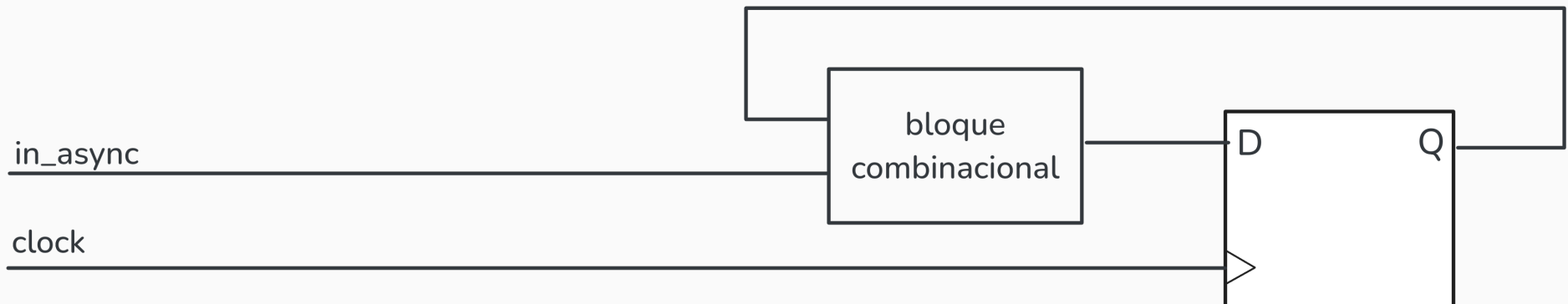
- Se dice que tenemos una *violación de tiempo* cuando la entrada cambia durante dicha ventana.
- La salida puede quedar en un estado **metaestable**.
- Eventualmente, la salida se estabilizará en un estado válido, luego de un *tiempo de resolución* T_r .
- T_r es una variable aleatoria, y no podemos predecir su valor.



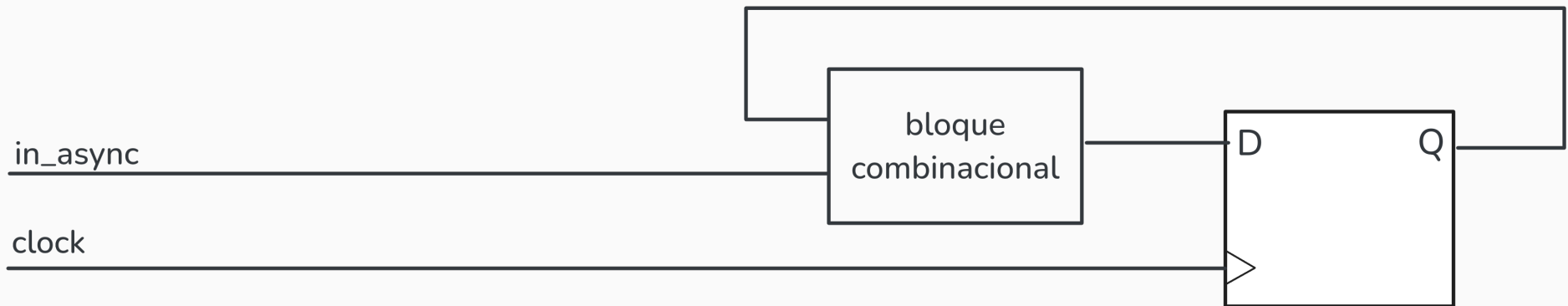
Violación de tiempo



- No estamos dando margen de resolución ($T_r = 0$)

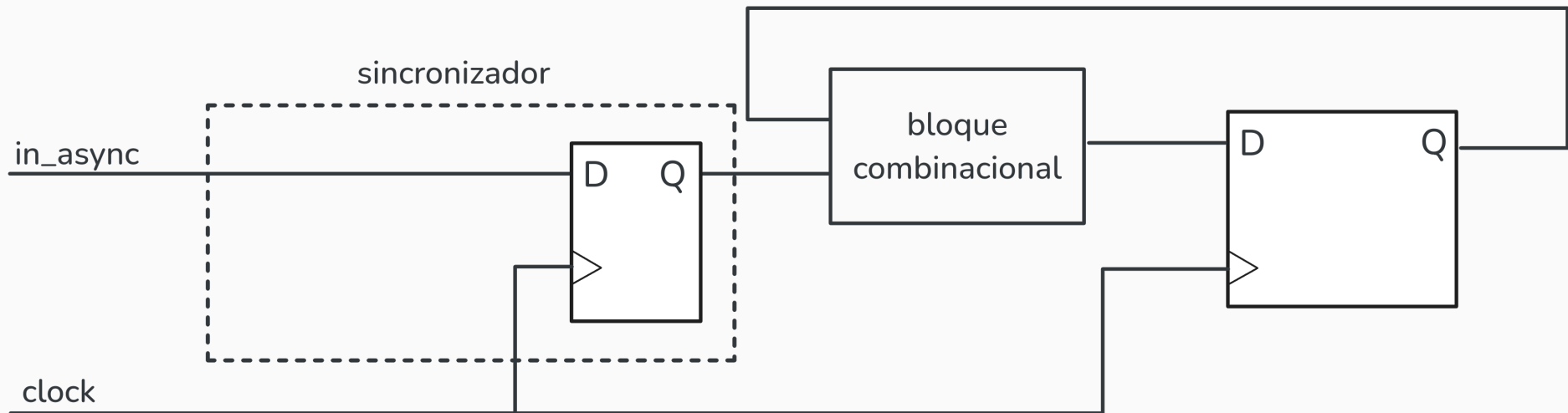


- No estamos dando margen de resolución ($T_r = 0$)
- MTBF (Mean Time Between Failures) es muy bajo



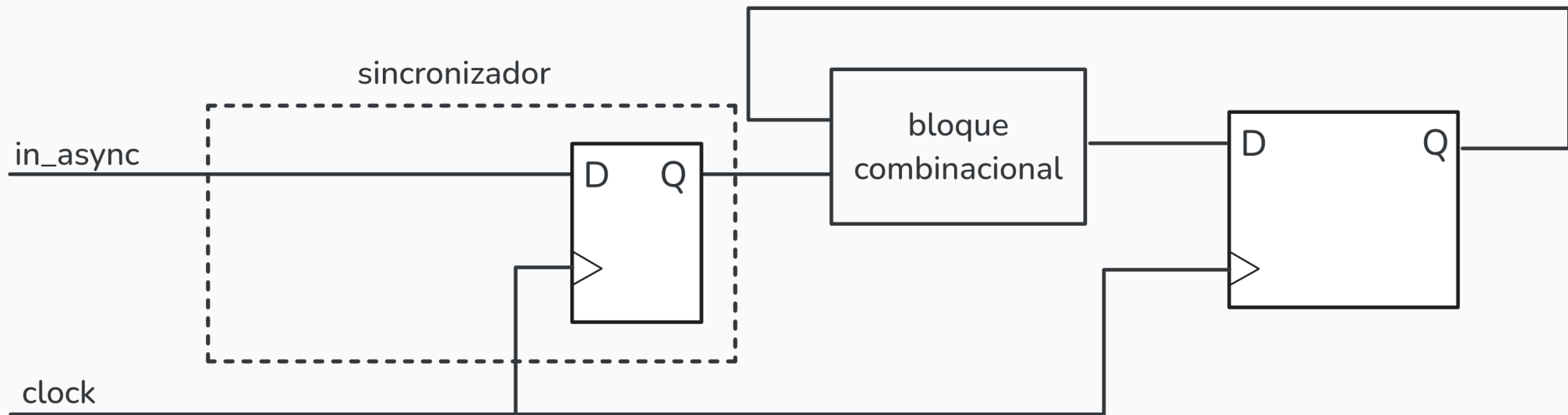
- No estamos dando margen de resolución ($T_r = 0$)
- MTBF (Mean Time Between Failures) es muy bajo
- Podemos agregar un segundo flip-flop para reducir la probabilidad de metaestabilidad

Metaestabilidad. Sincronizador simple



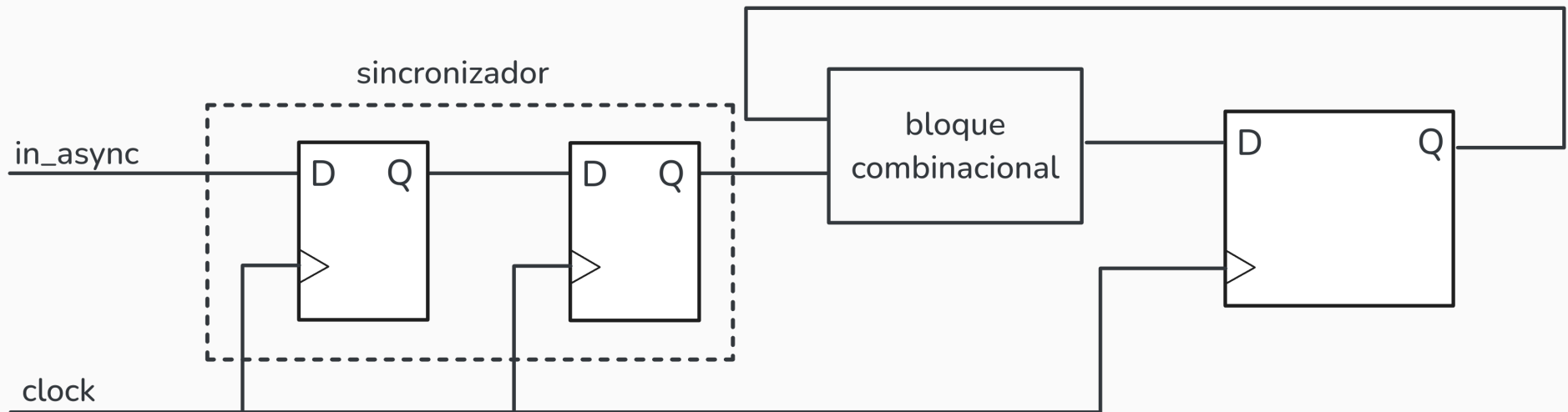
- $T_r = T_{\text{clk}} - (T_{\text{comb}} + T_{\text{setup}})$

Metaestabilidad. Sincronizador simple

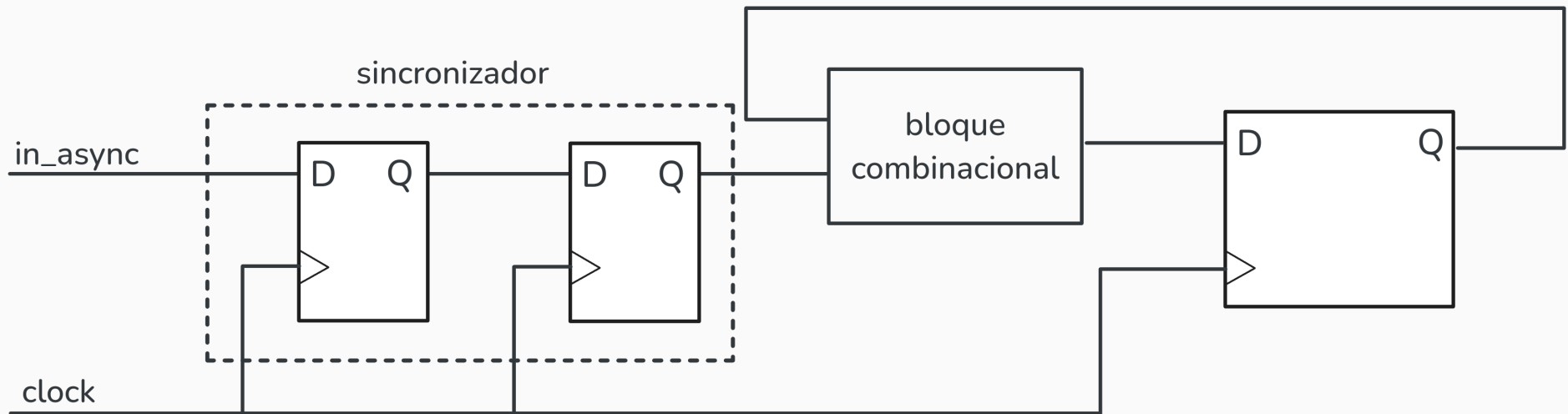


- $T_r = T_{\text{clk}} - (T_{\text{comb}} + T_{\text{setup}})$
- Aceptable si $T_{\text{comb}} \ll T_{\text{clk}}$

Metaestabilidad. Sincronizador doble



- $T_r = T_{\text{clk}} - T_{\text{setup}}$

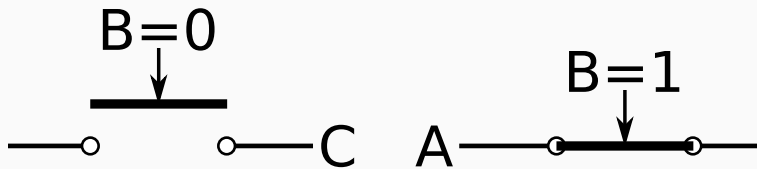


- $T_r = T_{\text{clk}} - T_{\text{setup}}$
- Esta solución es satisfactoria en la mayoría de las aplicaciones

Registros y memorias

Buffer de Tres Estados

Noción Eléctrica



Símbolo

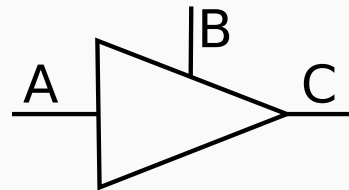
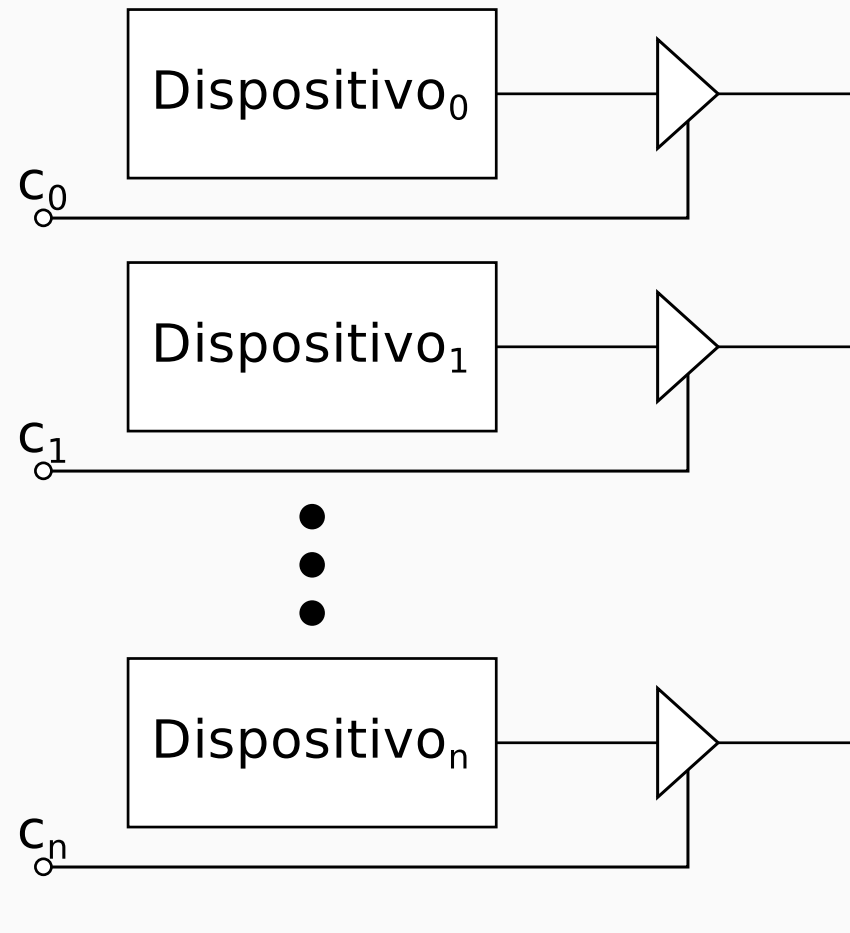


Tabla de Verdad

A	B	C
0	1	0
1	1	1
—	0	Hi-Z

Hi-Z significa “**alta impedancia**”, es decir, que tiene una resistencia alta al pasaje de corriente. Como consecuencia de esto, podemos considerar al pin C como desconectado del circuito.



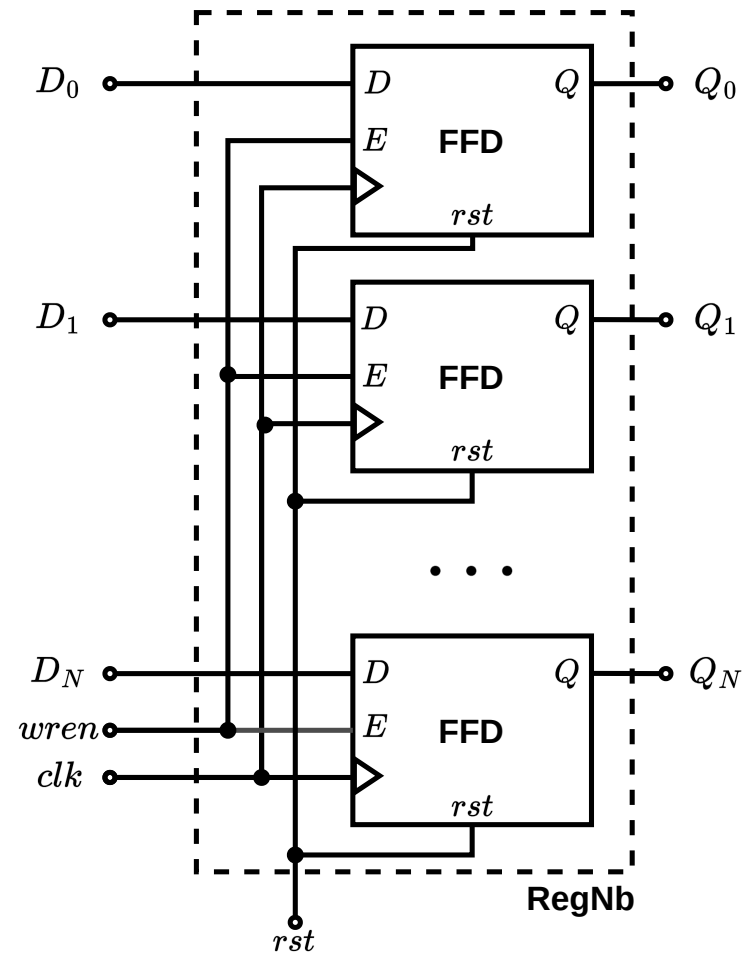
IMPORTANTE: Sólo deben ser usados a la salida de componentes para permitirles conectarse a un medio compartido (bus).

1. Diseñar un registro de N bits. El mismo debe contar con N entradas $D_0, \dots, D_N$ para ingresar el dato a almacenar, N salidas $Q_0, \dots, Q_N$ para ver el dato almacenado y las señales de control clk , rst y $wren$ para permitir la escritura del dato.

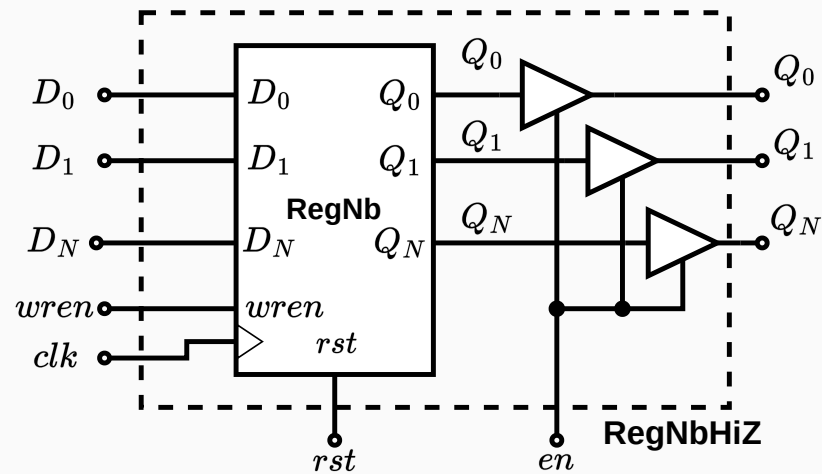
1. Diseñar un registro de N bits. El mismo debe contar con N entradas $D_0, \dots, D_N$ para ingresar el dato a almacenar, N salidas $Q_0, \dots, Q_N$ para ver el dato almacenado y las señales de control clk , rst y $wren$ para permitir la escritura del dato.
2. Modificar el diseño anterior agregándole componentes de 3 estados para que sólo cuando se active la señal de control en (*enable* o *habilitación*) muestre el dato almacenado.

1. Diseñar un registro de N bits. El mismo debe contar con N entradas $D_0, \dots, D_N$ para ingresar el dato a almacenar, N salidas $Q_0, \dots, Q_N$ para ver el dato almacenado y las señales de control clk , rst y $wren$ para permitir la escritura del dato.
2. Modificar el diseño anterior agregándole componentes de 3 estados para que sólo cuando se active la señal de control en (*enable* o *habilitación*) muestre el dato almacenado.
3. Modificar nuevamente el diseño para que D_i y Q_i estén conectadas entre sí al mismo tiempo, teniendo en lugar de N entradas y N salidas, N entrada-salidas.

Solución - Ejercicio 1.a

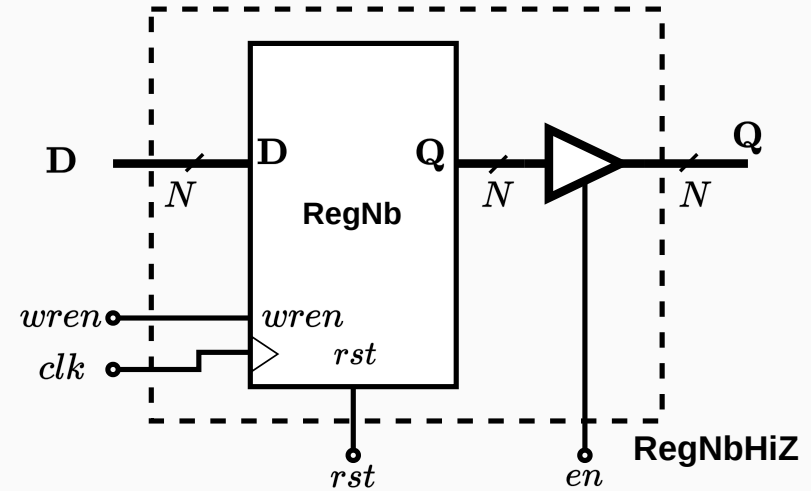
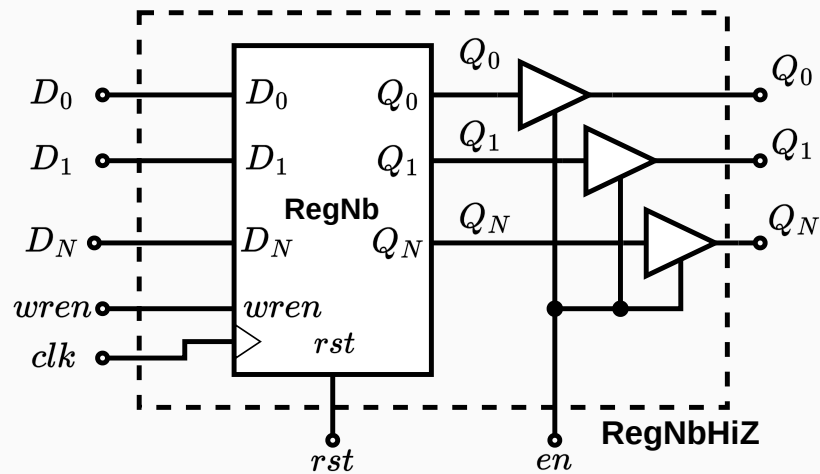


Usamos el componente anterior:

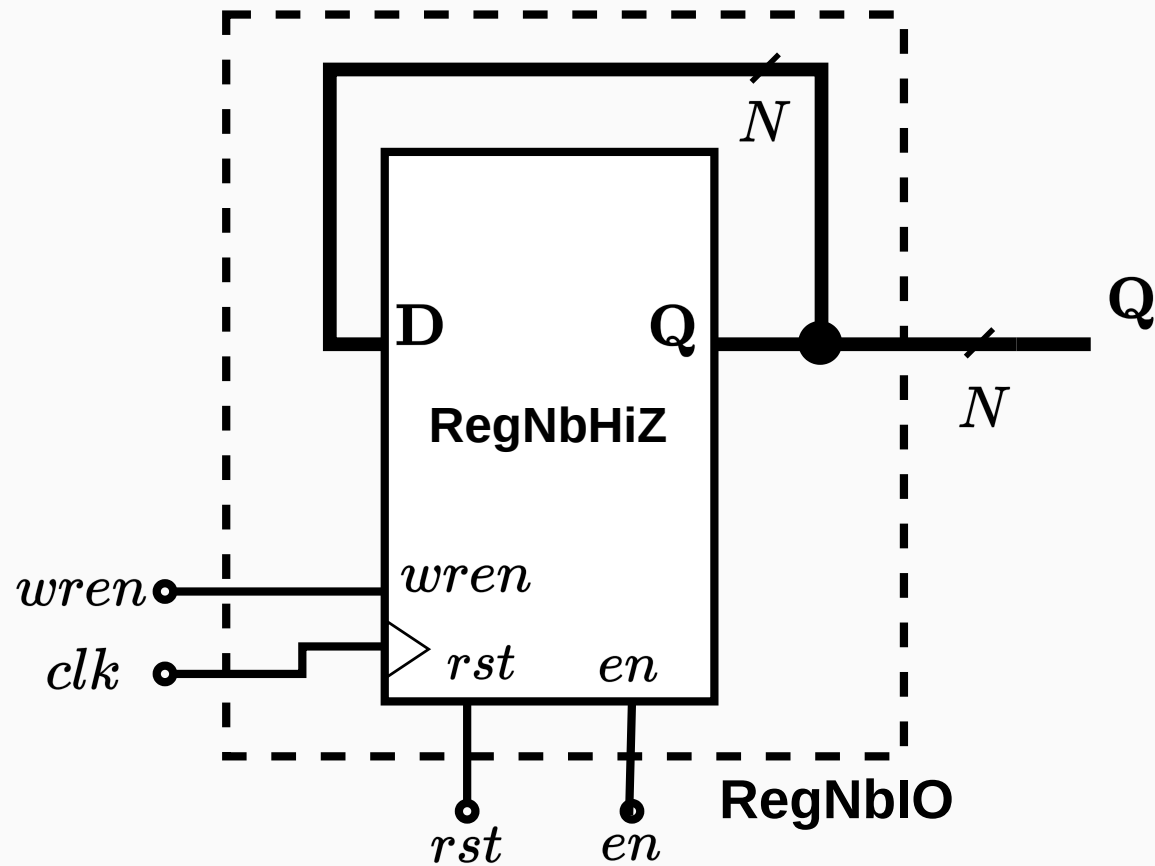


Solución - Ejercicio 1.b

Usamos el componente anterior:



Más prolijo :)





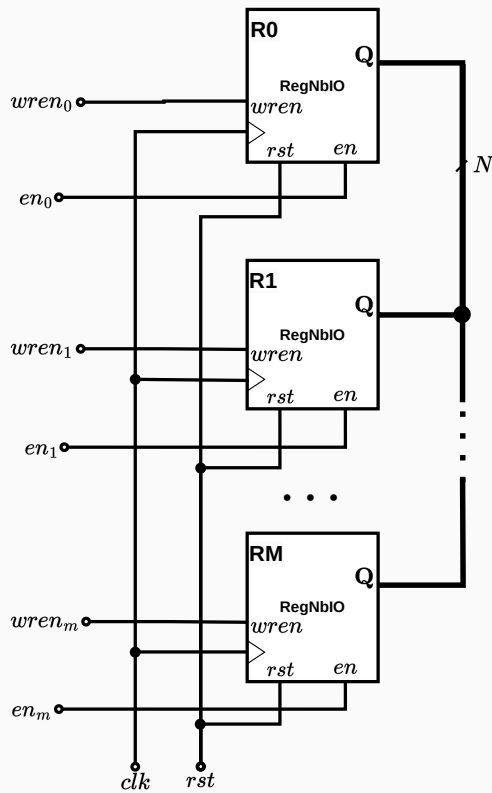
1. Realizar el esquema de interconexión de M registros como el diseñado en el ejercicio anterior.

1. Realizar el esquema de interconexión de M registros como el diseñado en el ejercicio anterior.
2. Dar una secuencia de valores de las señales de control para que se copie el dato del R_1 al R_0 .

DEPARTAMENTO
DE COMPUTACION

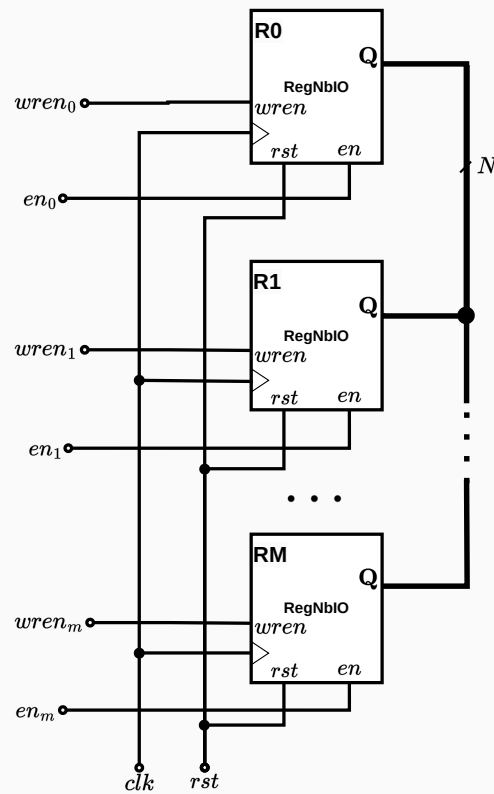
Facultad de Ciencias Exactas y Naturales - UBA

-



Ejercicio 2

1. Realizar el esquema de interconexión de M registros como el diseñado en el ejercicio anterior.
2. Dar una secuencia de valores de las señales de control para que se copie el dato del R_1 al R_0 .

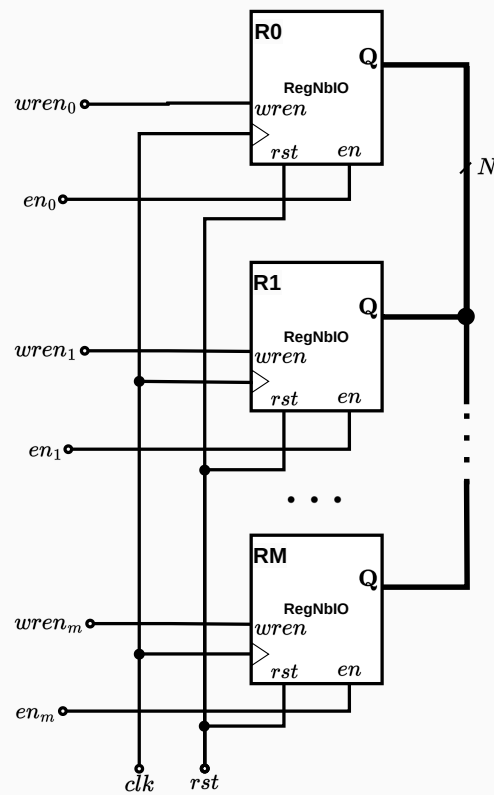


Señales de control:

R0	R1	...	RM
$wren_0$	$wren_1$	...	$wren_M$
rst	rst	...	rst
en_0	en_1	...	en_M

Ejercicio 2

1. Realizar el esquema de interconexión de M registros como el diseñado en el ejercicio anterior.
2. Dar una secuencia de valores de las señales de control para que se copie el dato del R_1 al R_0 .



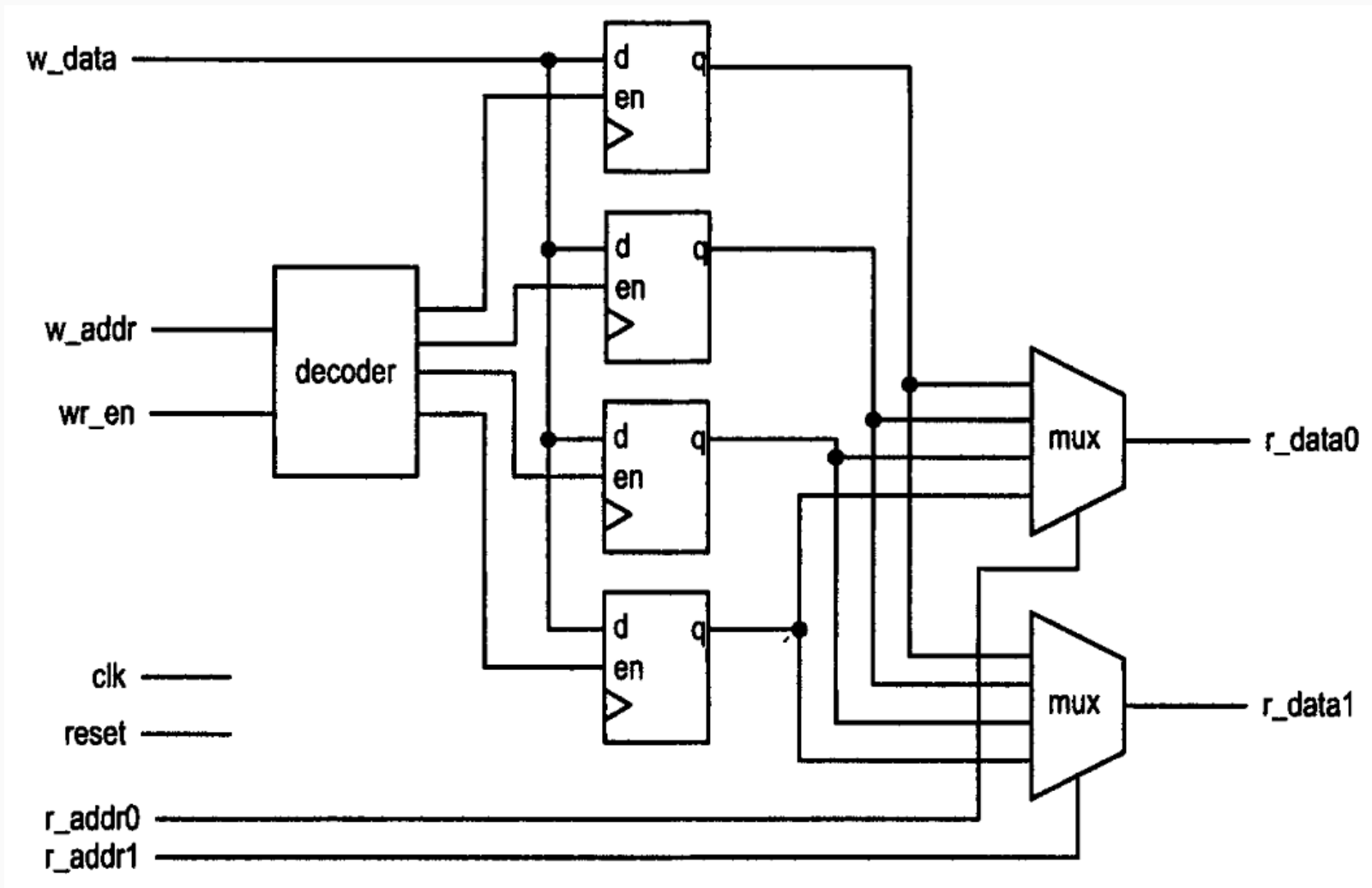
Señales de control:

R0	R1	...	RM
wren ₀	wren ₁	...	wren _M
rst	rst	...	rst
en ₀	en ₁	...	en _M

Inician todas las señales en 0. Luego se sigue la siguiente secuencia:

- $en_1 \leftarrow 1$
- $wren_0 \leftarrow 1$
- ...clk....
- $wren_0 \leftarrow 0$
- $en_1 \leftarrow 0$

Ahora podemos tener varios registros de tamaño N y seleccionar de cuál leer o escribir.



Register file (SV)

```
module reg_file #(
    parameter int DATA_WIDTH = 32,
    parameter int ADDR_WIDTH = 5
)()
    input logic          clk,
    input logic          rst,
    // Puerto A (Read/Write con índice compartido regA_idx)
    input logic [ADDR_WIDTH-1:0] regA_idx,
    input logic [DATA_WIDTH-1:0] regA_din,
    output logic [DATA_WIDTH-1:0] regA_dout,
    input logic          regA_we,
    // Puerto B (Read/Write con índice compartido regB_idx)
    input logic [ADDR_WIDTH-1:0] regB_idx,
    input logic [DATA_WIDTH-1:0] regB_din,
    output logic [DATA_WIDTH-1:0] regB_dout,
    input logic          regB_we
);

localparam int NUM_REGS = 1 << ADDR_WIDTH;
```

SV


```
// Arreglo de registros
logic [DATA_WIDTH-1:0] rf [0:NUM_REGS-1];

// Lectura sincrónica Read-First: se lee el contenido previo en
regA_idx y regB_idx
// antes de que cualquier escritura modifique la posición de memoria en
el flanco.
always_ff @(posedge clk or posedge rst) begin
    if (rst) begin
        regA_dout <= '0;
        regB_dout <= '0;
    end else begin
        regA_dout <= (regA_idx == '0) ? '0 : rf[regA_idx];
        regB_dout <= (regB_idx == '0) ? '0 : rf[regB_idx];
    end
end

// Escritura sincrónica en Puerto A y Puerto B (descarta escrituras en
el registro 0)
```

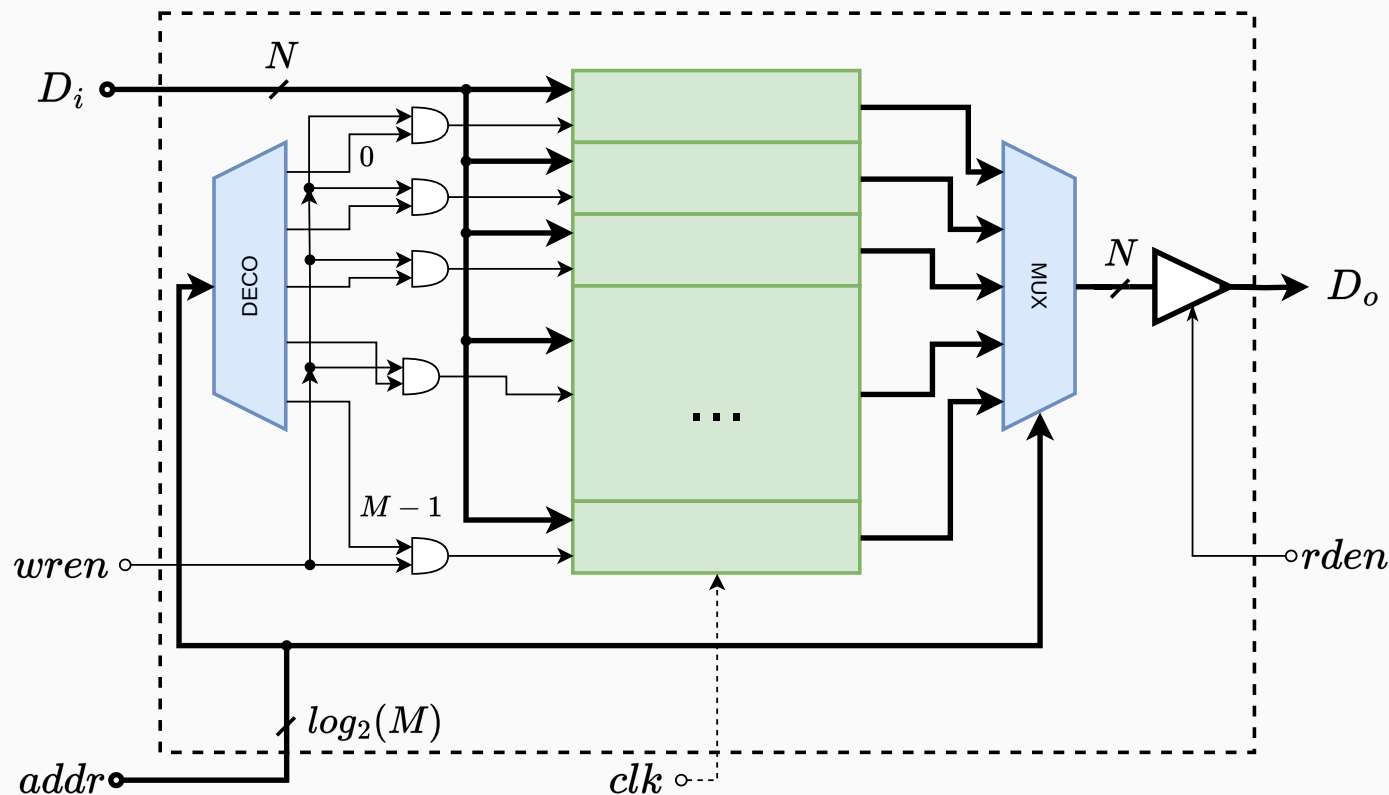
Register file (SV)

```
always_ff @(posedge clk) begin
    if (regA_we && (regA_idx != '0)) begin
        rf[regA_idx] <= regA_din;
    end
    if (regB_we && (regB_idx != '0)) begin
        rf[regB_idx] <= regB_din;
    end
end

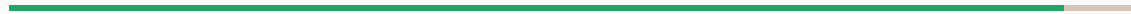
endmodule
```

Extendiendo la idea, **conceptualmente** podemos pensar una memoria como M posiciones de almacenamiento de N bits cada una.

Extendiendo la idea, **conceptualmente** podemos pensar una memoria como M posiciones de almacenamiento de N bits cada una.



Conclusiones



- Estudiamos la realimentación en circuitos para mantener un dato en el tiempo y vimos implementaciones de *latches*

- Estudiamos la realimentación en circuitos para mantener un dato en el tiempo y vimos implementaciones de *latches*
- Estudiamos los problemas asociados a los tiempos de propagación de las señales



- Estudiamos la realimentación en circuitos para mantener un dato en el tiempo y vimos implementaciones de *latches*
- Estudiamos los problemas asociados a los tiempos de propagación de las señales
- Analizamos el uso de un *clock* para limitar la cantidad de estados, controlar las transiciones y evitar carreras

- Estudiamos la realimentación en circuitos para mantener un dato en el tiempo y vimos implementaciones de *latches*
- Estudiamos los problemas asociados a los tiempos de propagación de las señales
- Analizamos el uso de un *clock* para limitar la cantidad de estados, controlar las transiciones y evitar carreras
- Vimos algunas implementaciones de flip-flops, registros y memorias

Básica

- *Digital Design and Computer Architecture*, David Harris, Sarah Harris
- *Digital Design: Principles and Practices*, John F. Wakerly

Avanzada

- *RTL Hardware Design Using VHDL*, Pong P. Chu

¿Preguntas?
